

# API文档

[跳转附录](#)

## 约定

### 路由前缀

- 规范前缀（推荐写法）：
  - 账户：`/account/*`
  - 好友：`/friend/*`
  - 会话/群聊/消息：`/chat/*`
- 兼容前缀（与 `/chat/*` 等价）：`/new/*`、`/message/*`（历史遗留别名）

### 鉴权

- HTTP API：`Authorization: Bearer <JWT Token>`
- WebSocket：`ws://<host>/ws/chat?token=<JWT Token>`

### 返回格式

- 成功：HTTP 200，固定包含 `{"code": 0, "info": "Succeed."}`，并在同一层级返回该接口的业务字段（例如 `jwt_token`、`friends`、`messages` 等；部分接口会使用 `data` 字段承载主要数据）。
- 失败：HTTP 非 200，`{"code": <业务码>, "info": "..."}`

说明：为便于阅读，本文部分成功响应示例可能省略固定字段 `info`，以实际返回为准。

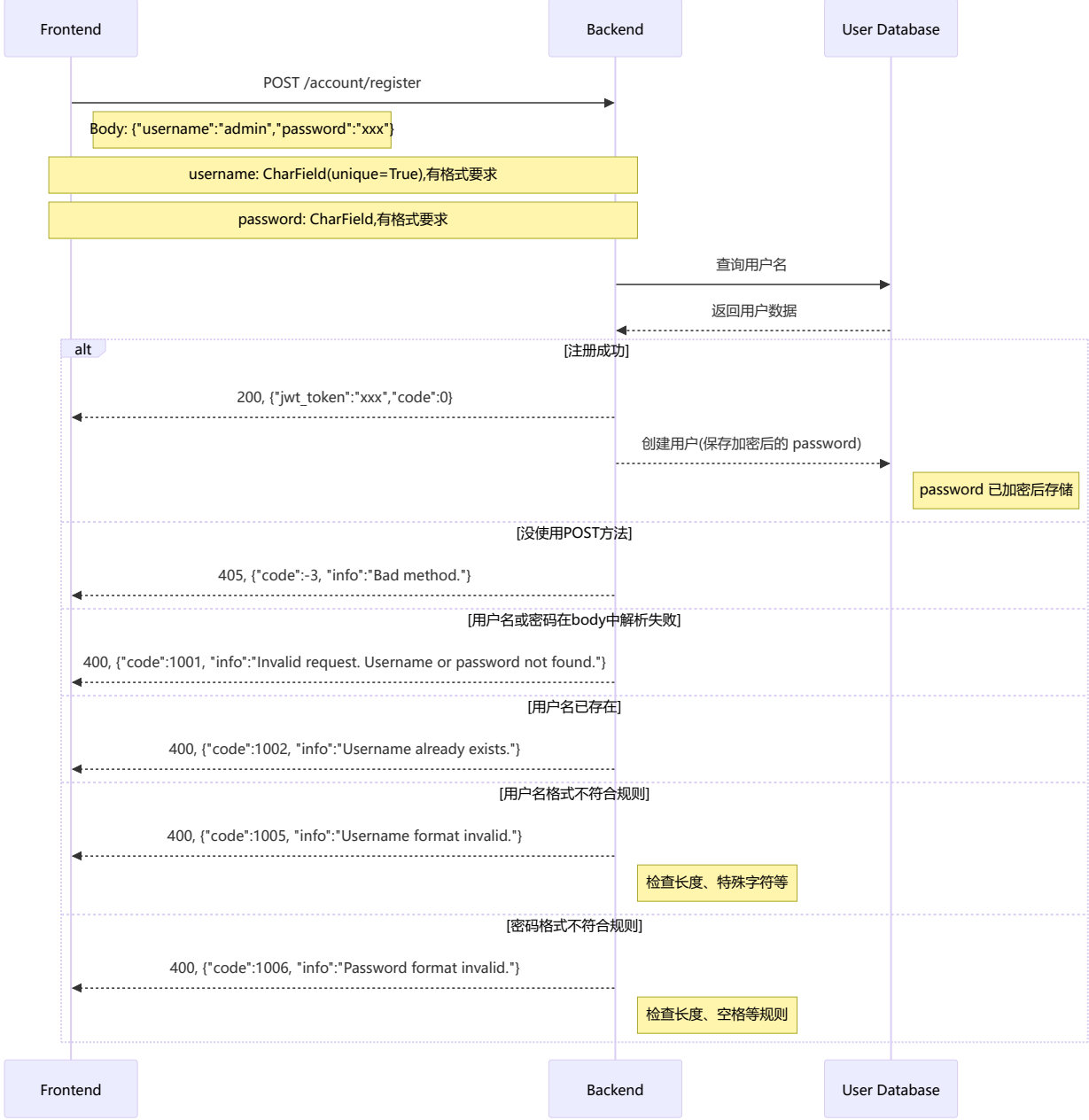
## 一、用户管理部分

### 1、用户注册（基本管理）

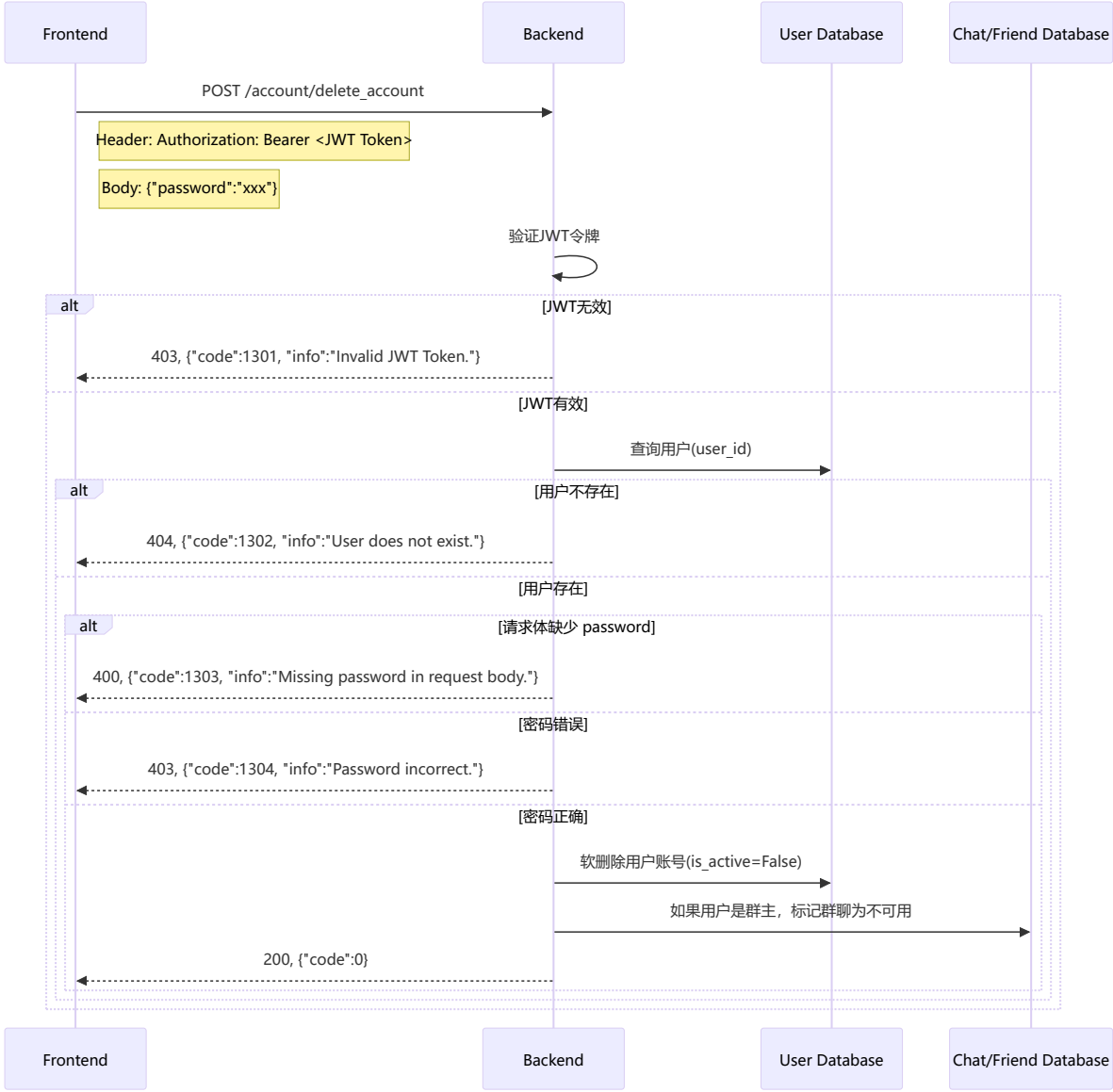
格式规定：

【username】1-30字符，只能包含汉字、字母、数字、下划线、点（不能以后两个开头或结尾）

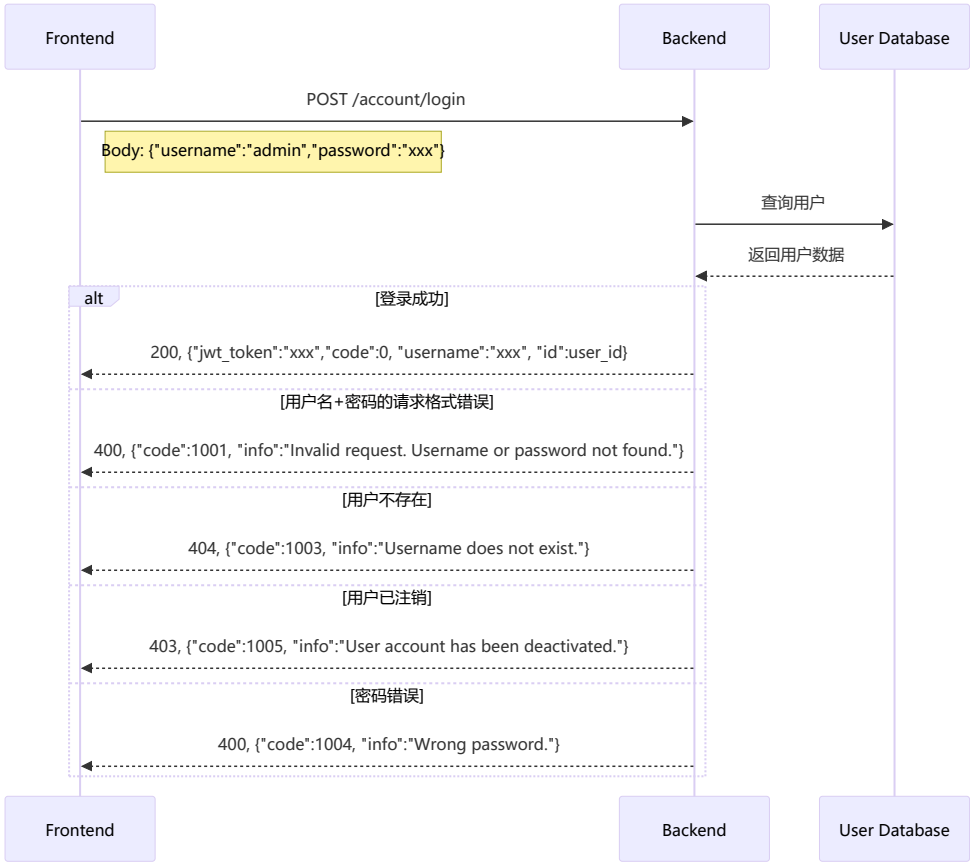
【password】8-128字符，必须包含字母和数字，不能包含空格



2、用户注销（基本管理）

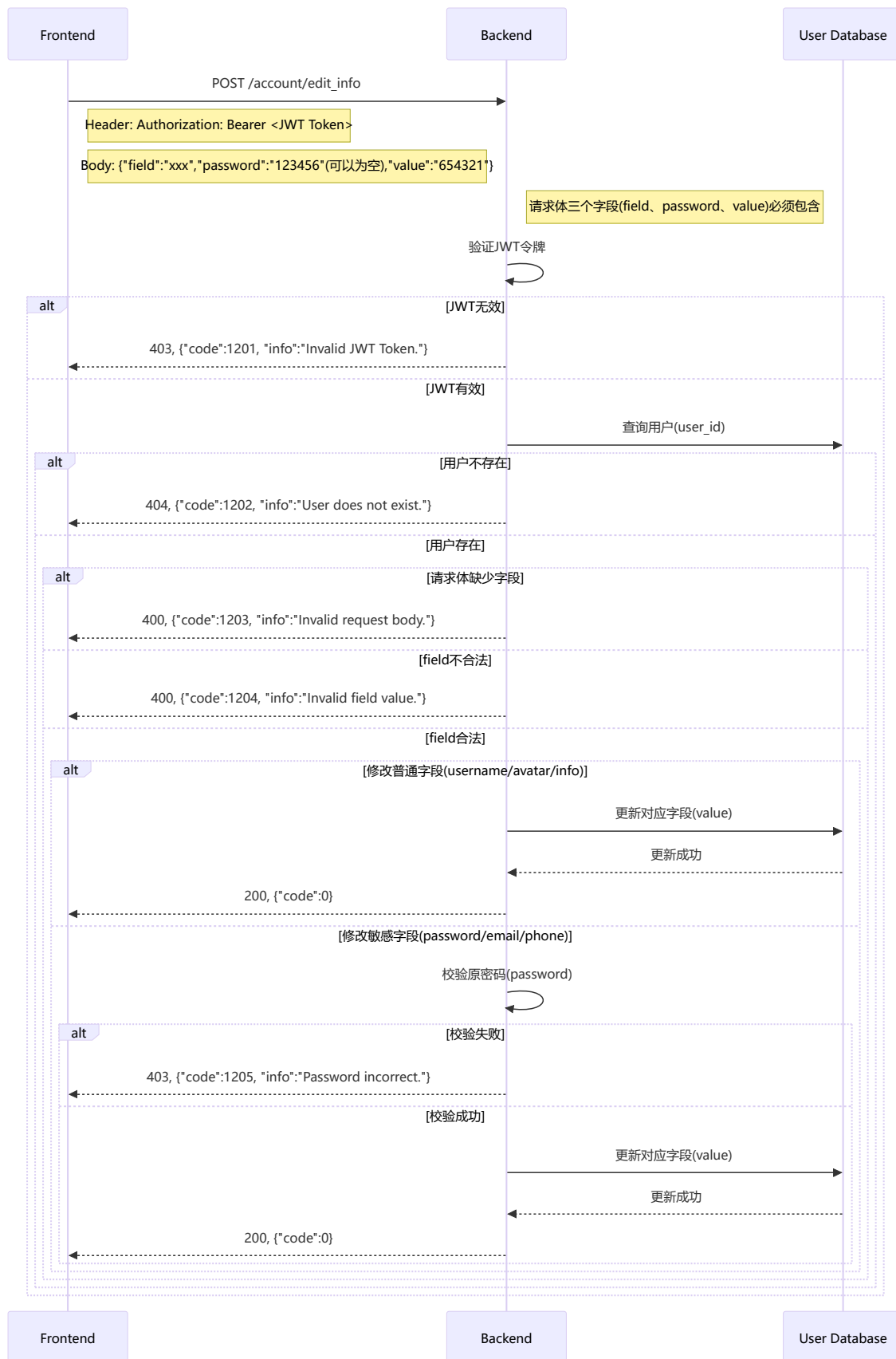


3、登录登出（用户认证）



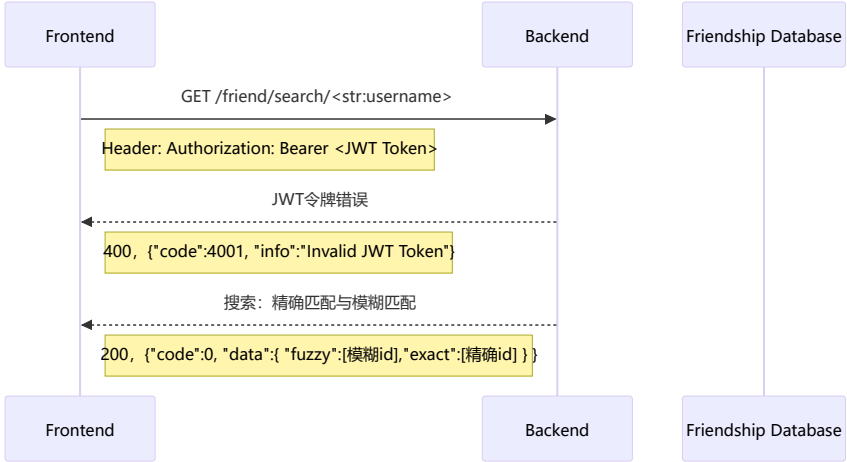
4、信息编辑（用户认证）

修改个人信息



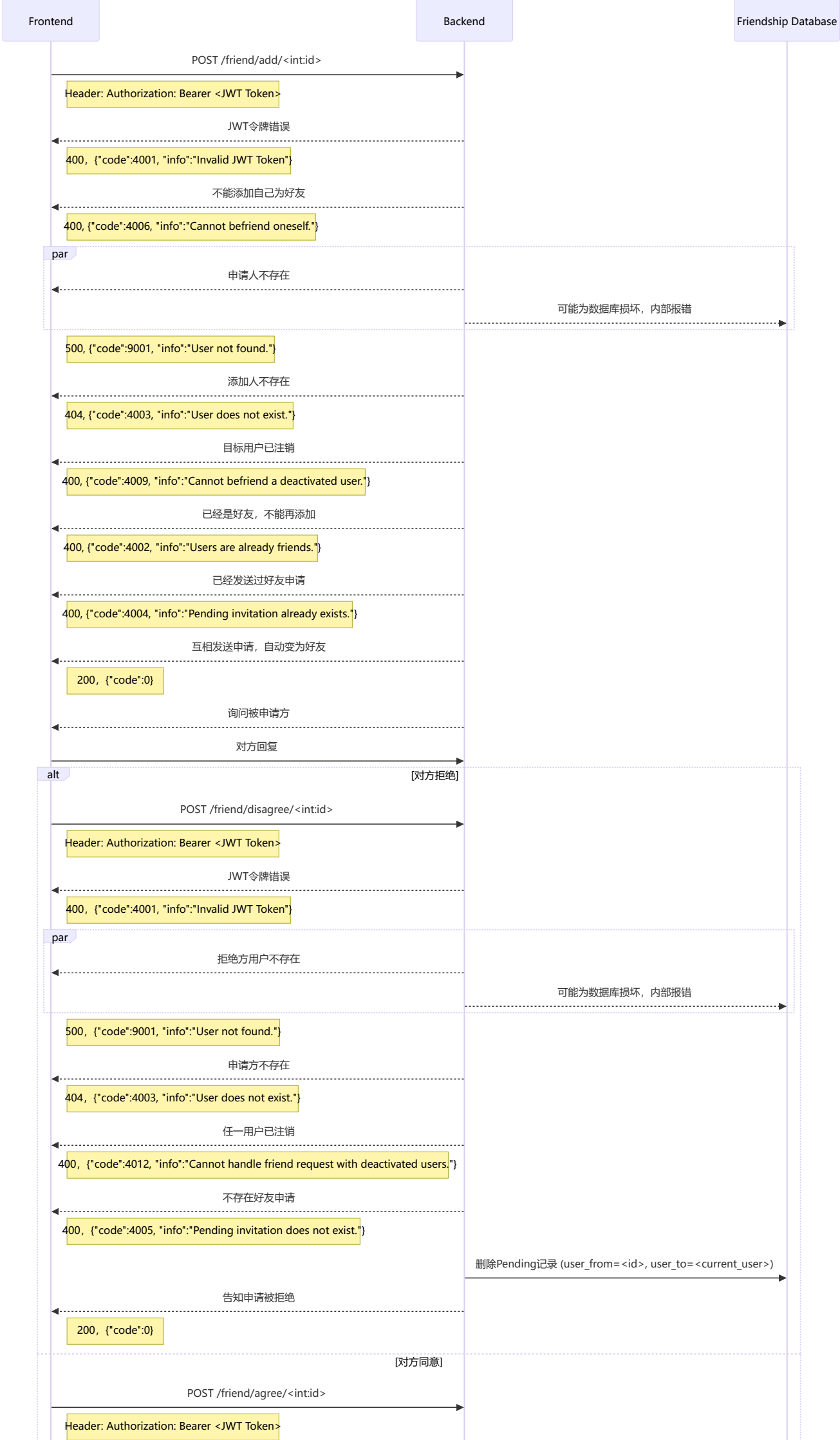
## 5、用户查找（好友关系）

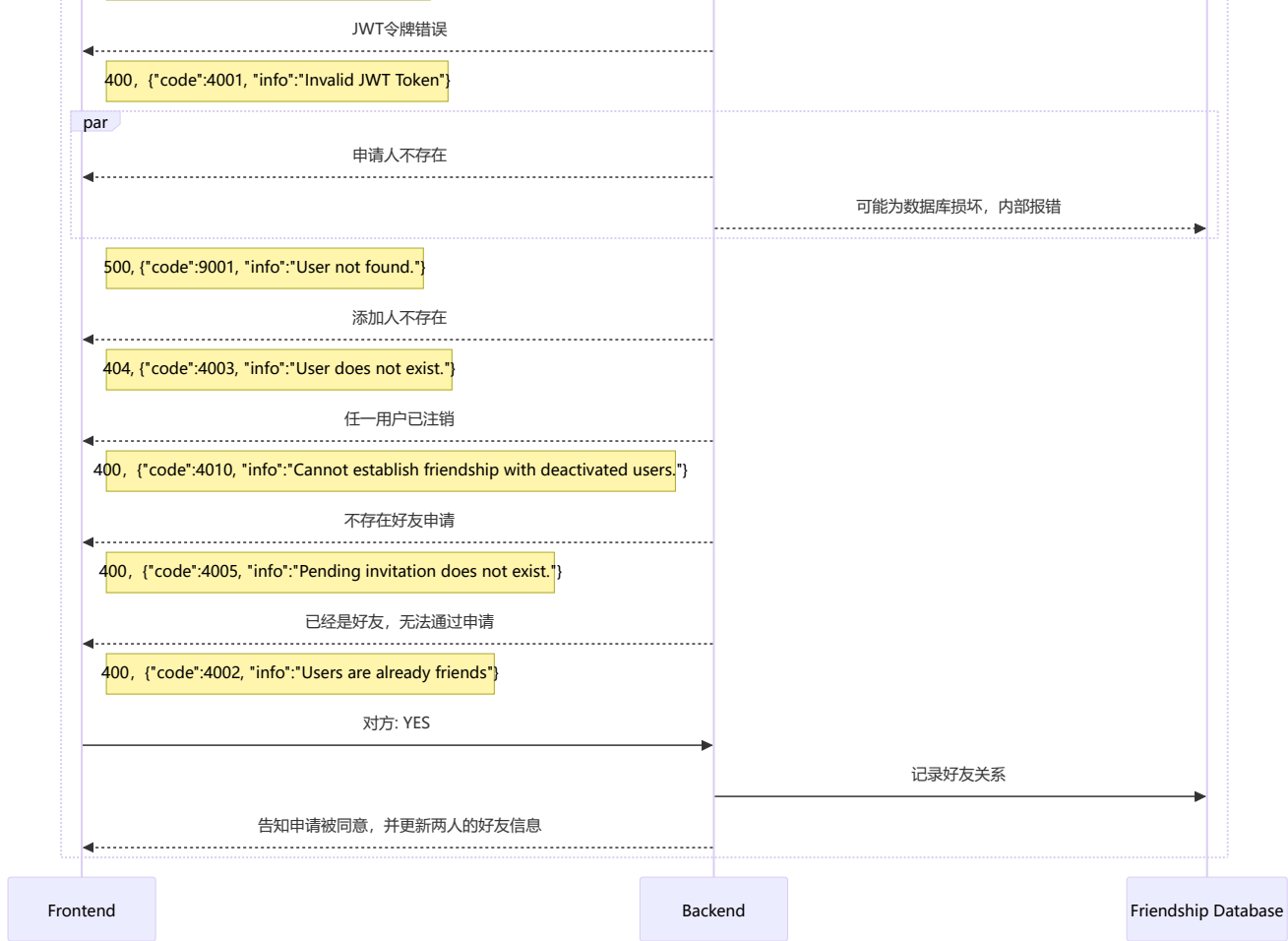
查找用户



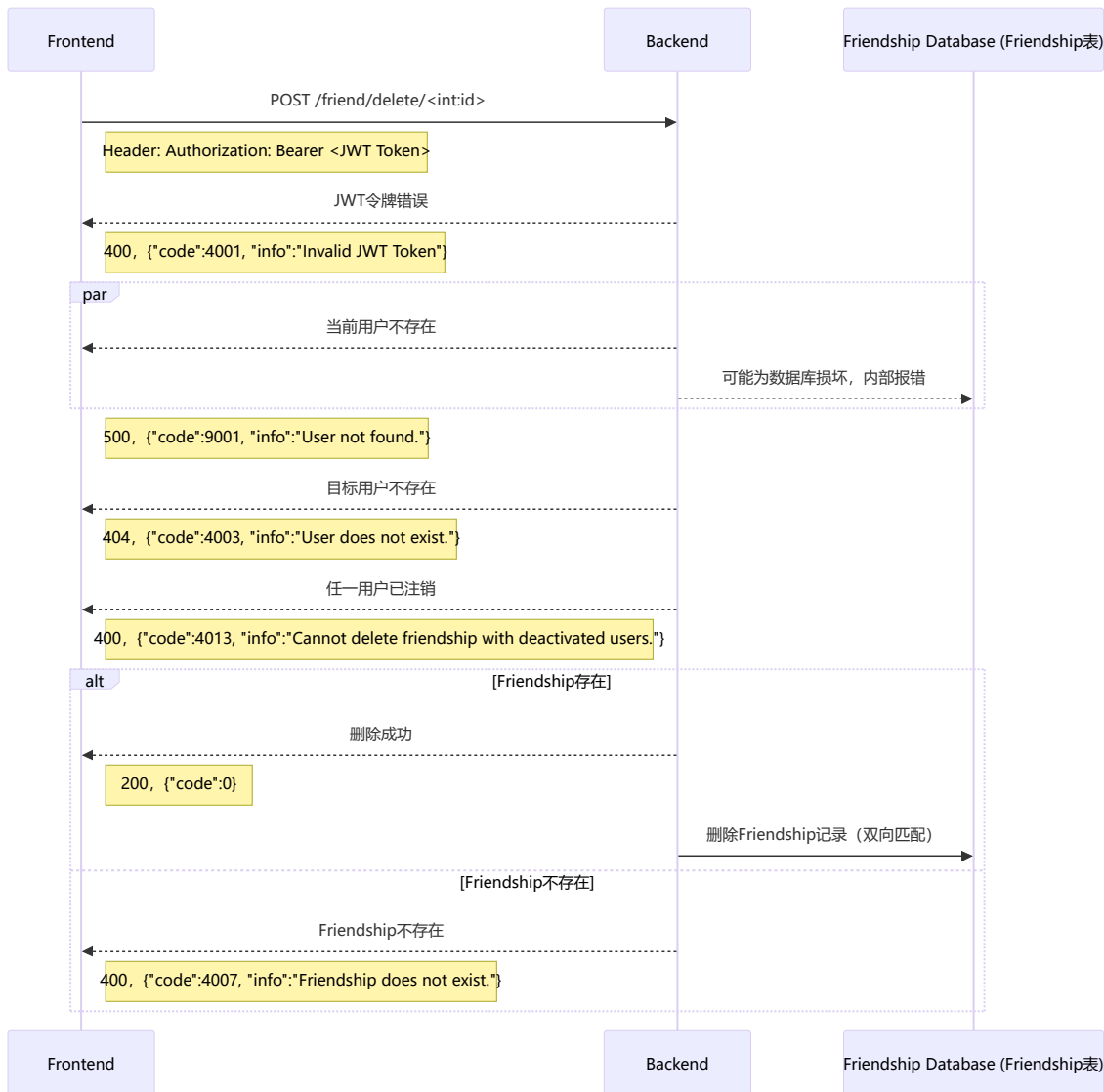
## 6、好友申请（好友关系）

(申请添加好友+同意拒绝)



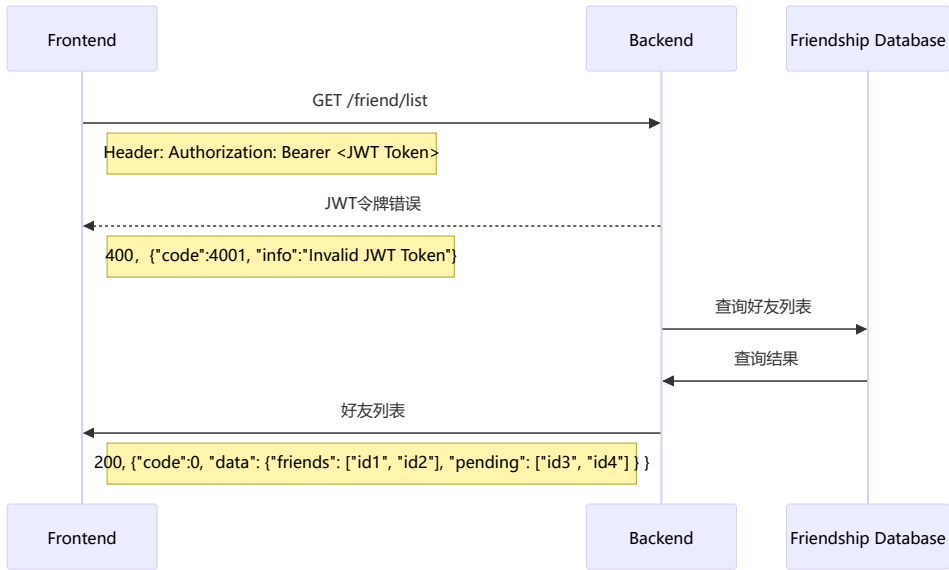


## 7、好友删除（好友关系）



## 8、好友列表

获取好友列表



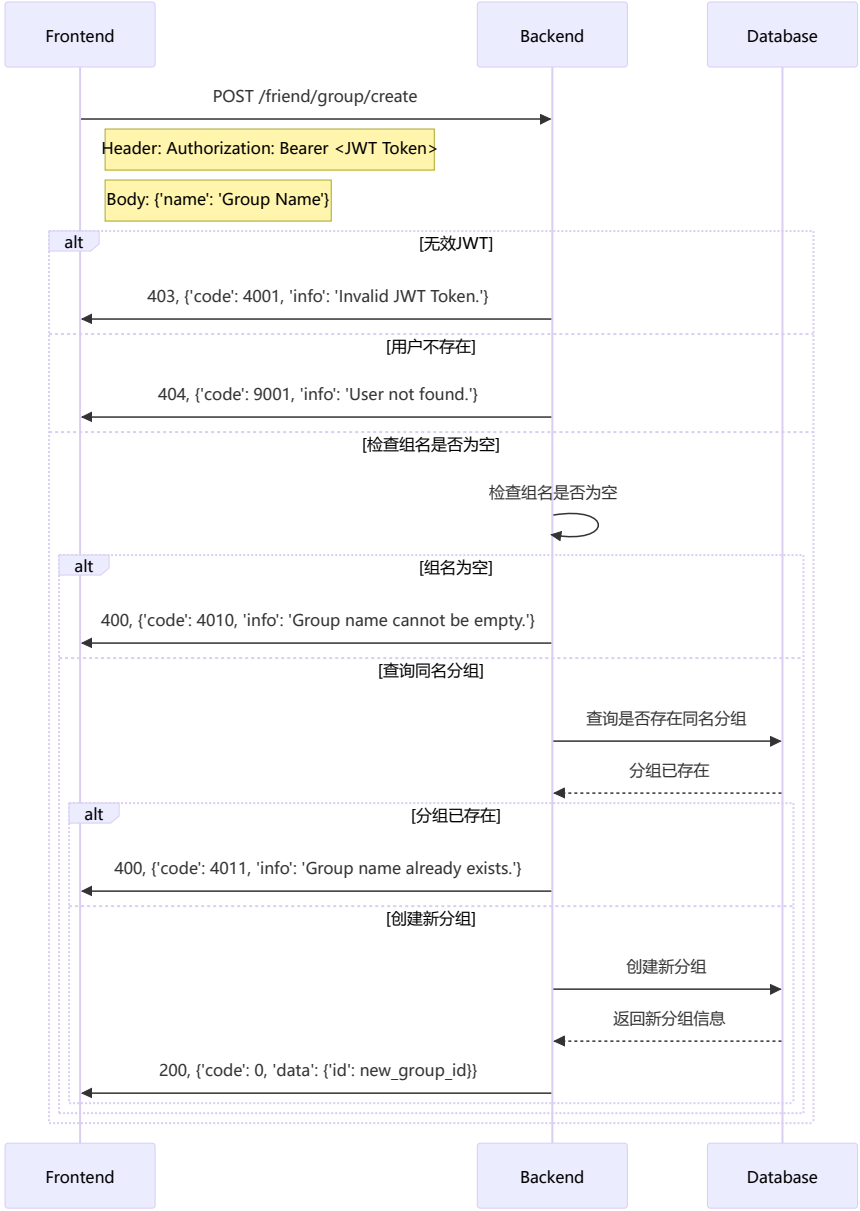
9、检查好友关系



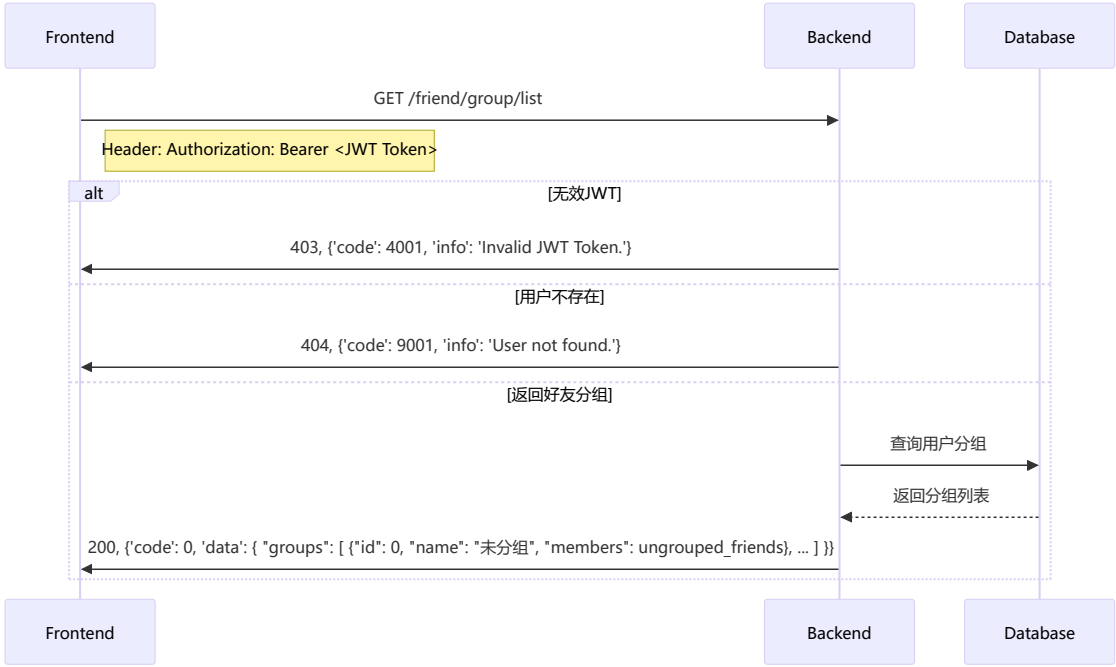
10、好友分组

创建好友分组

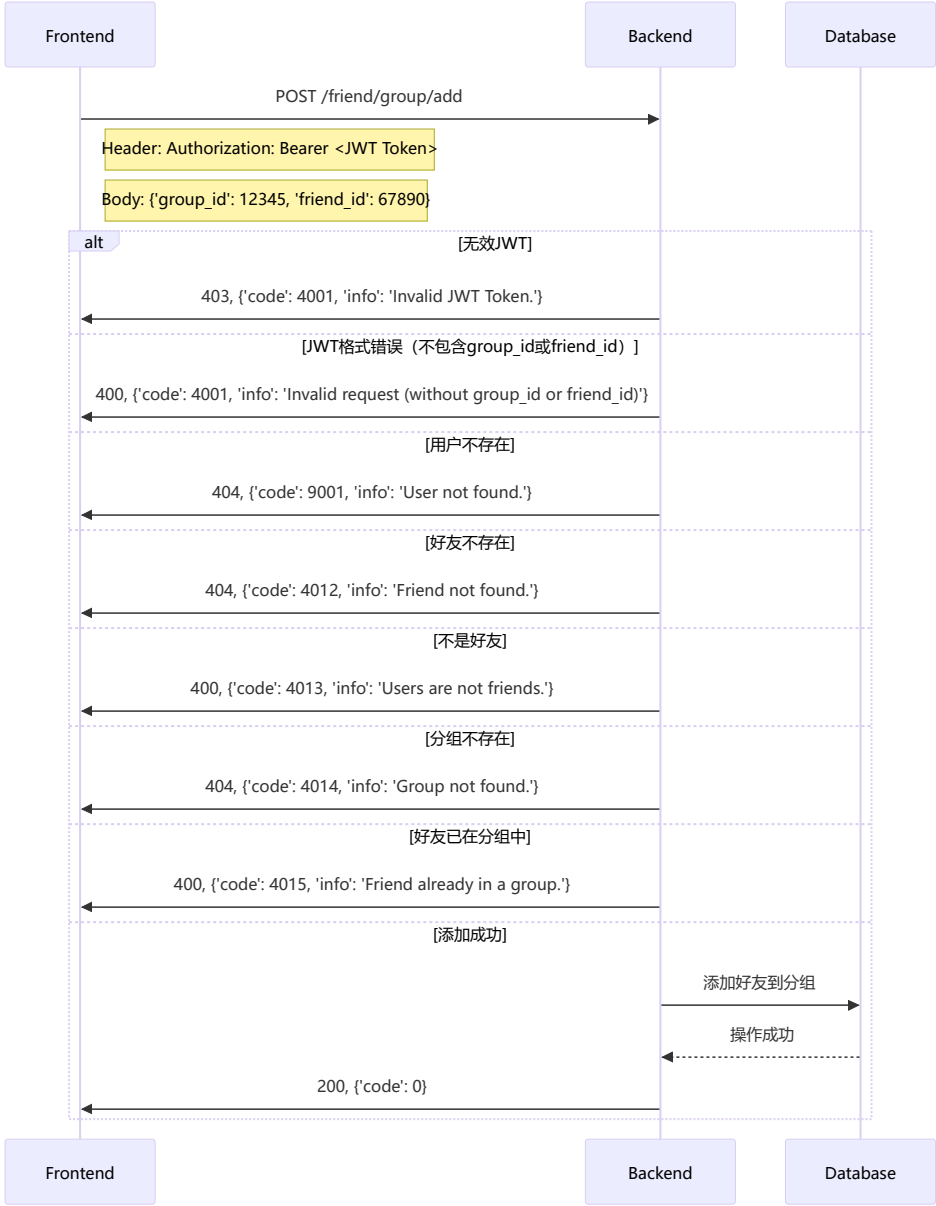




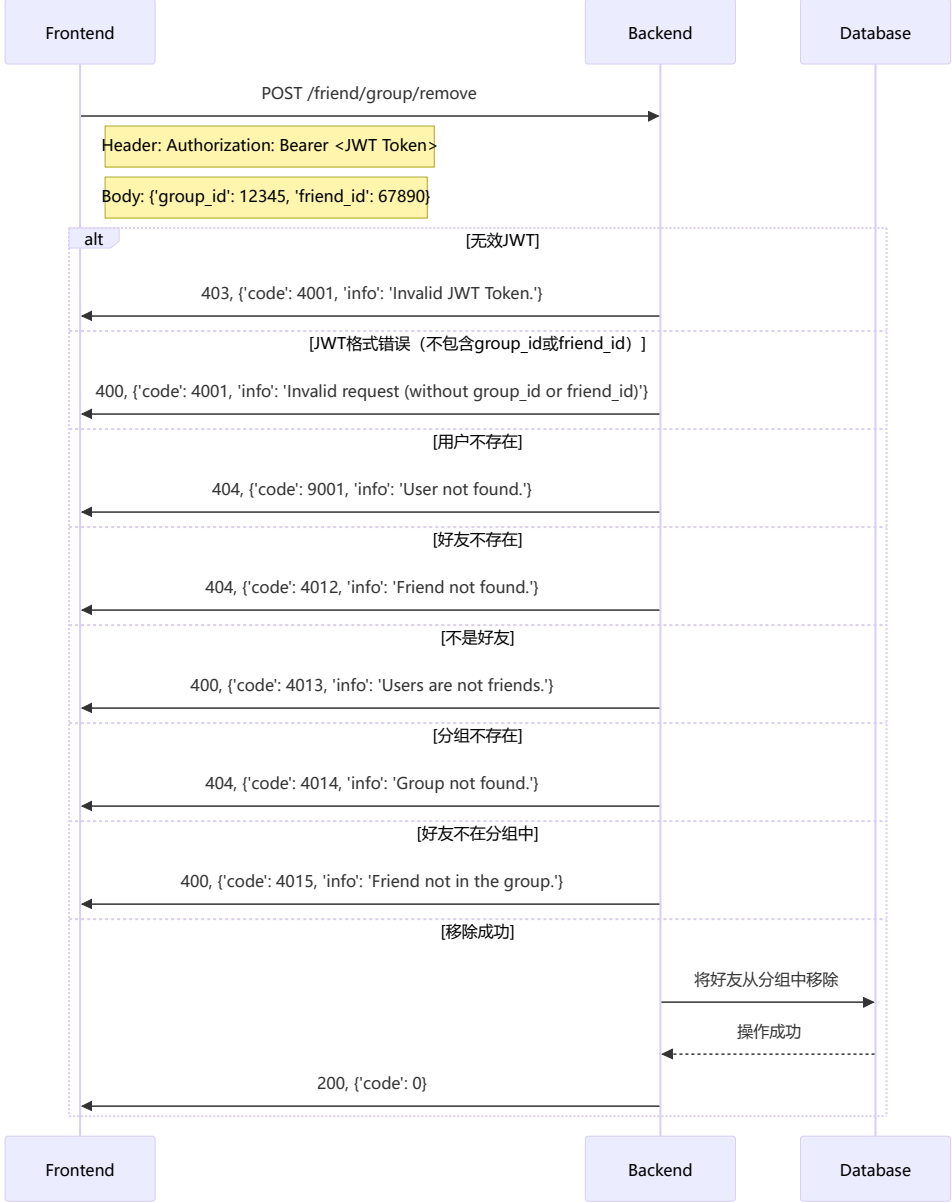
列出好友分组



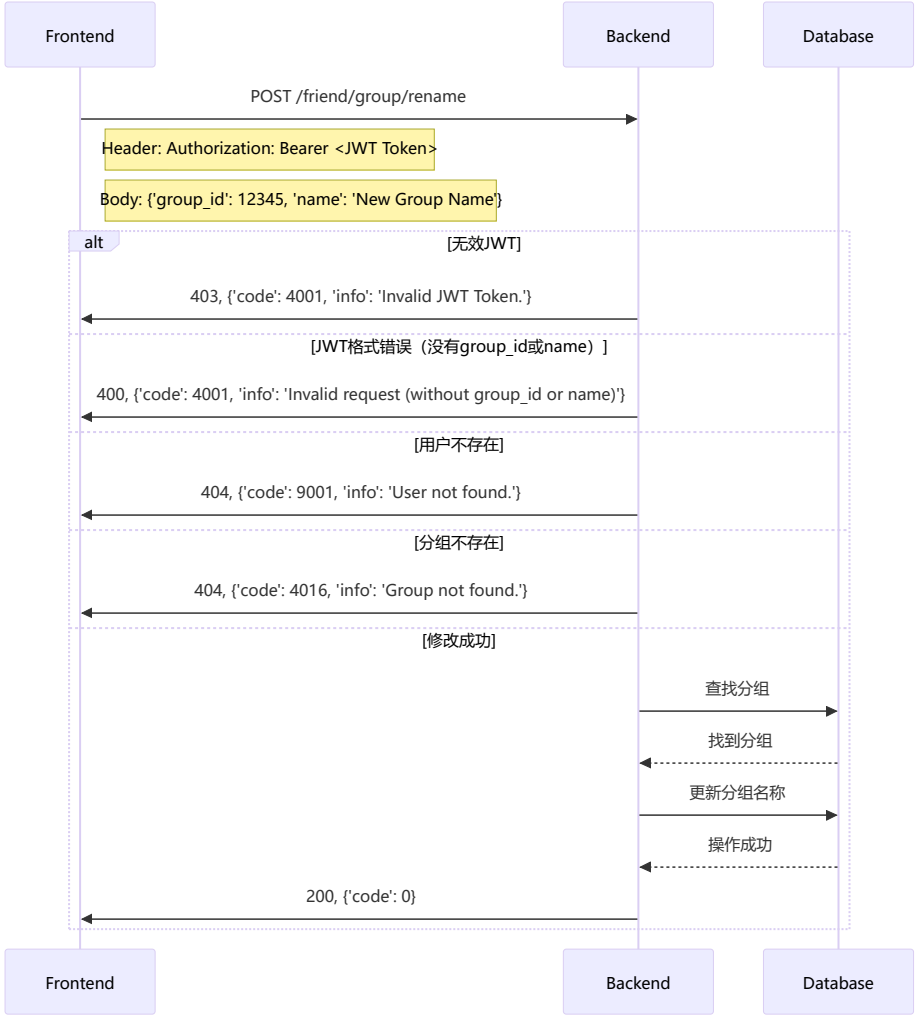
把好友添加到分组里



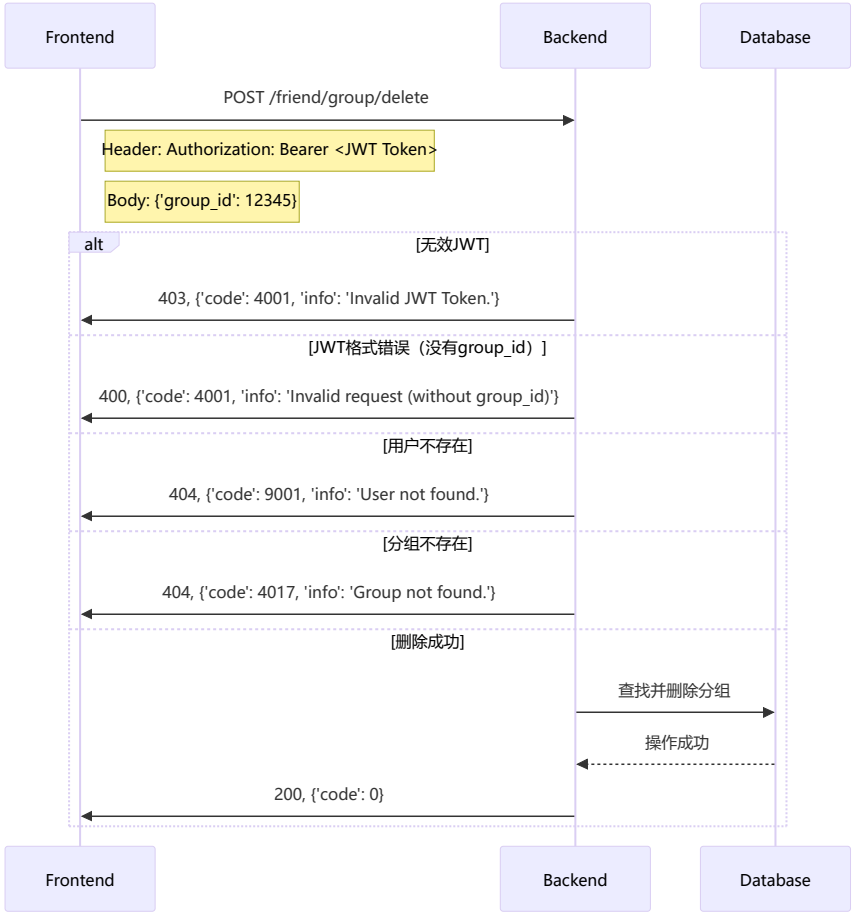
从分组移除好友



重命名分组

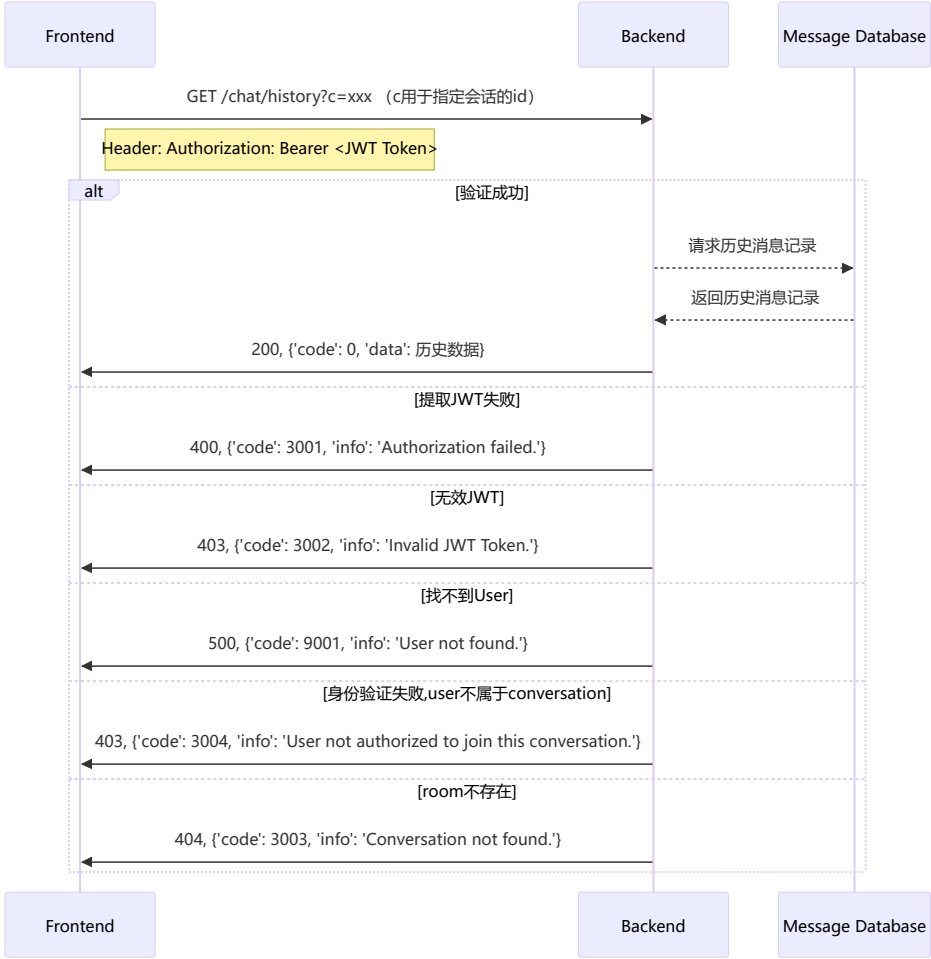


删除分组

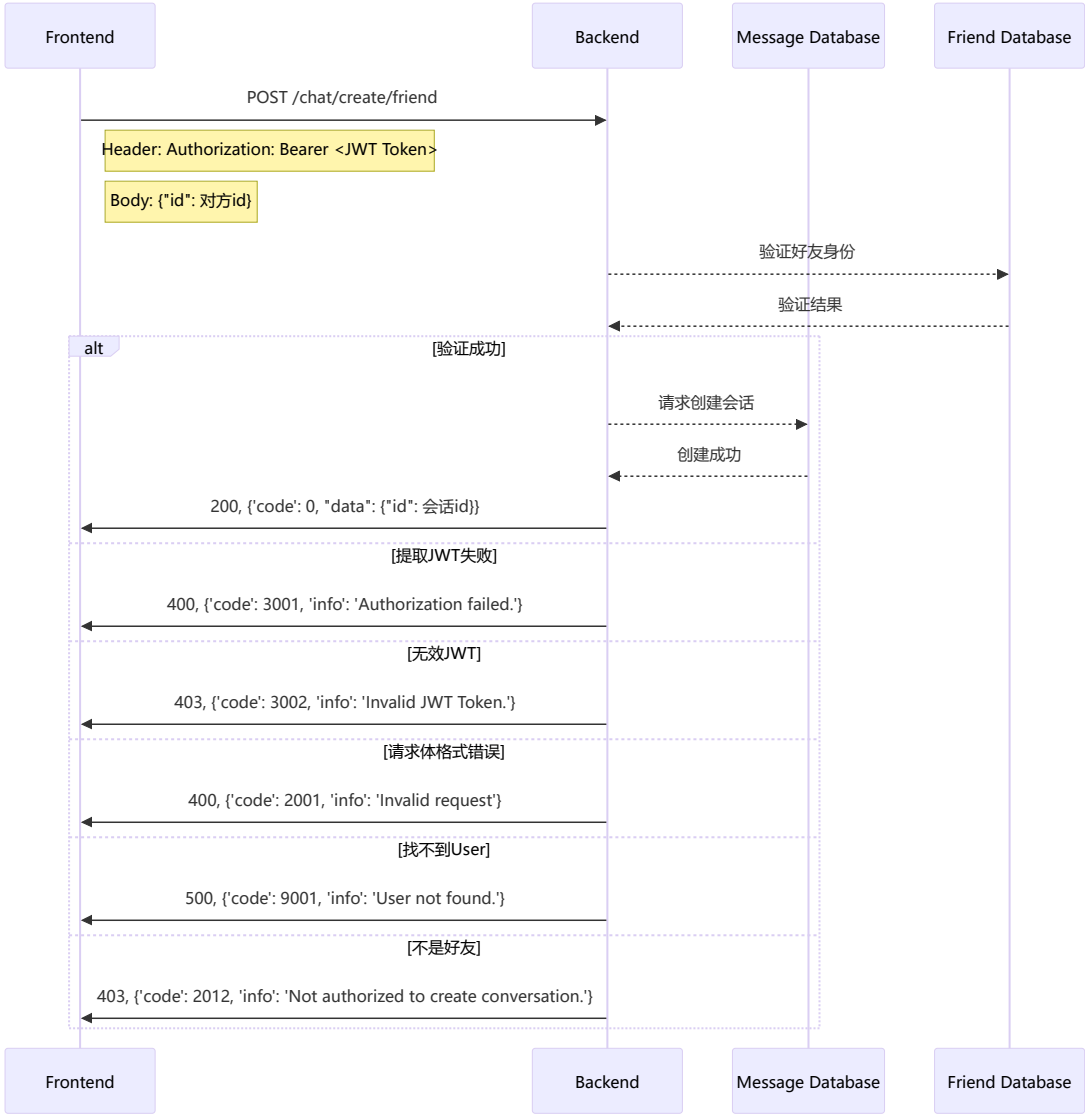


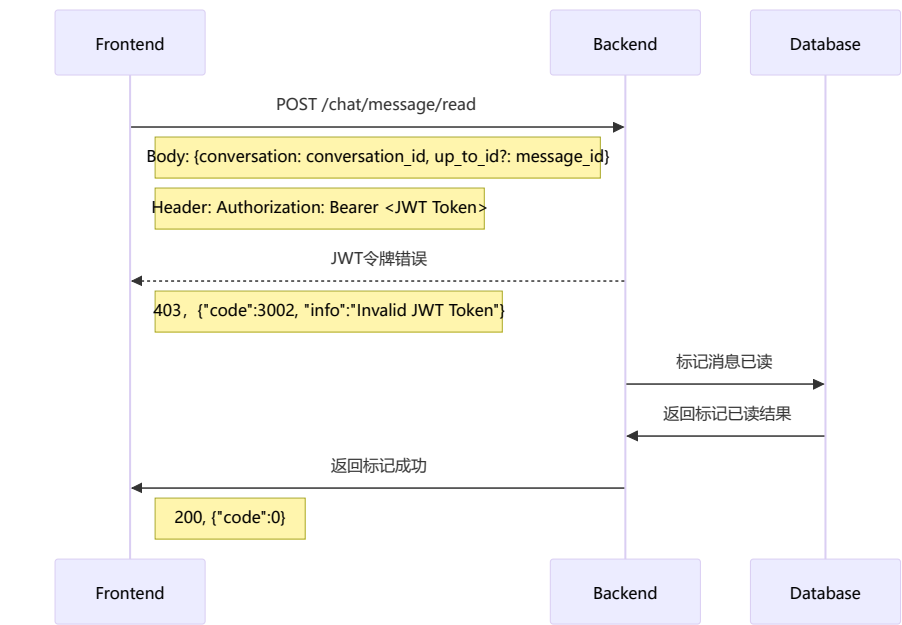
二、在线会话通用部分（25分）

请求消息记录

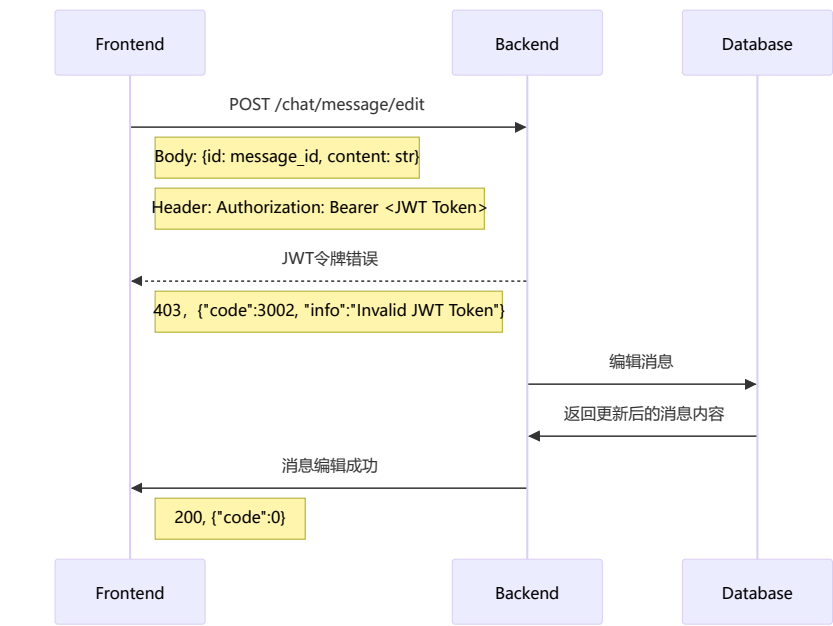


请求创建好友会话

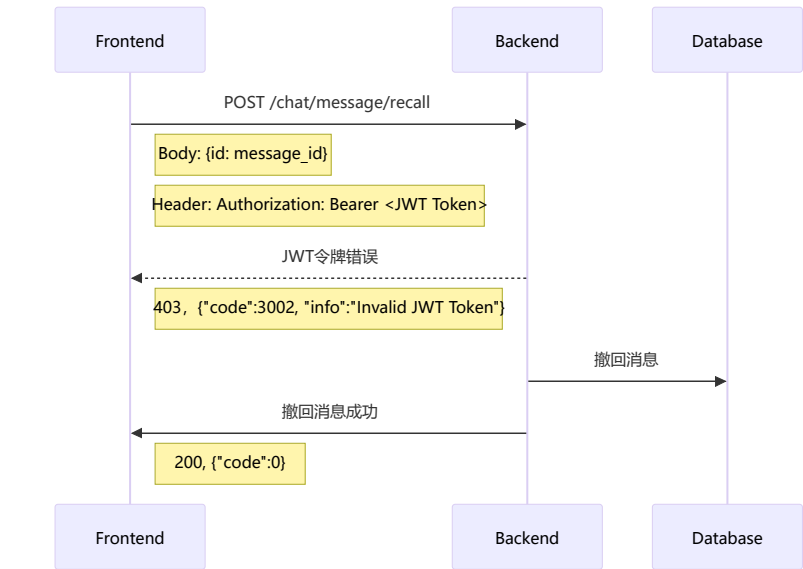




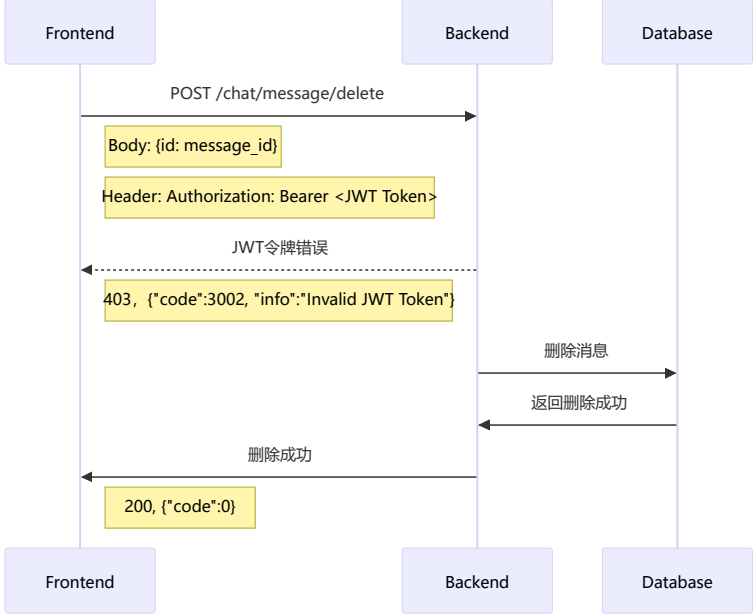
## 编辑消息



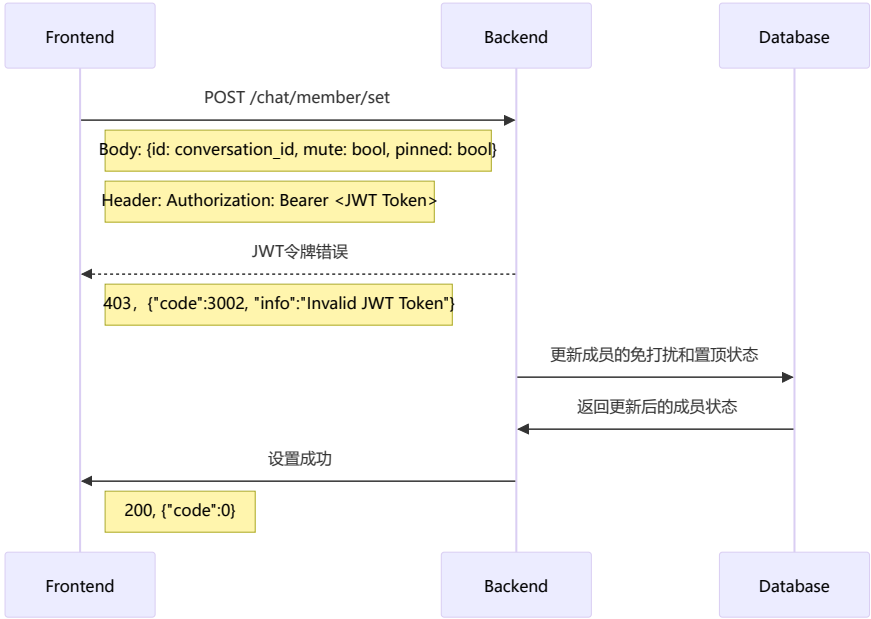
## 撤回消息



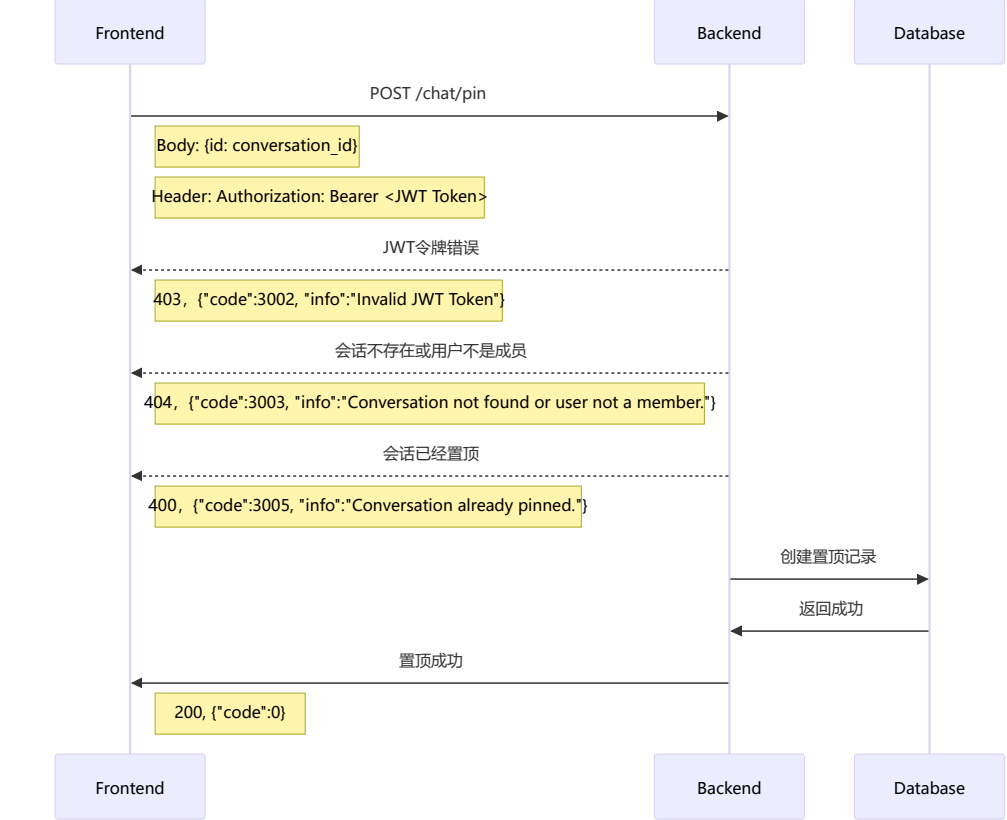
## 删除消息



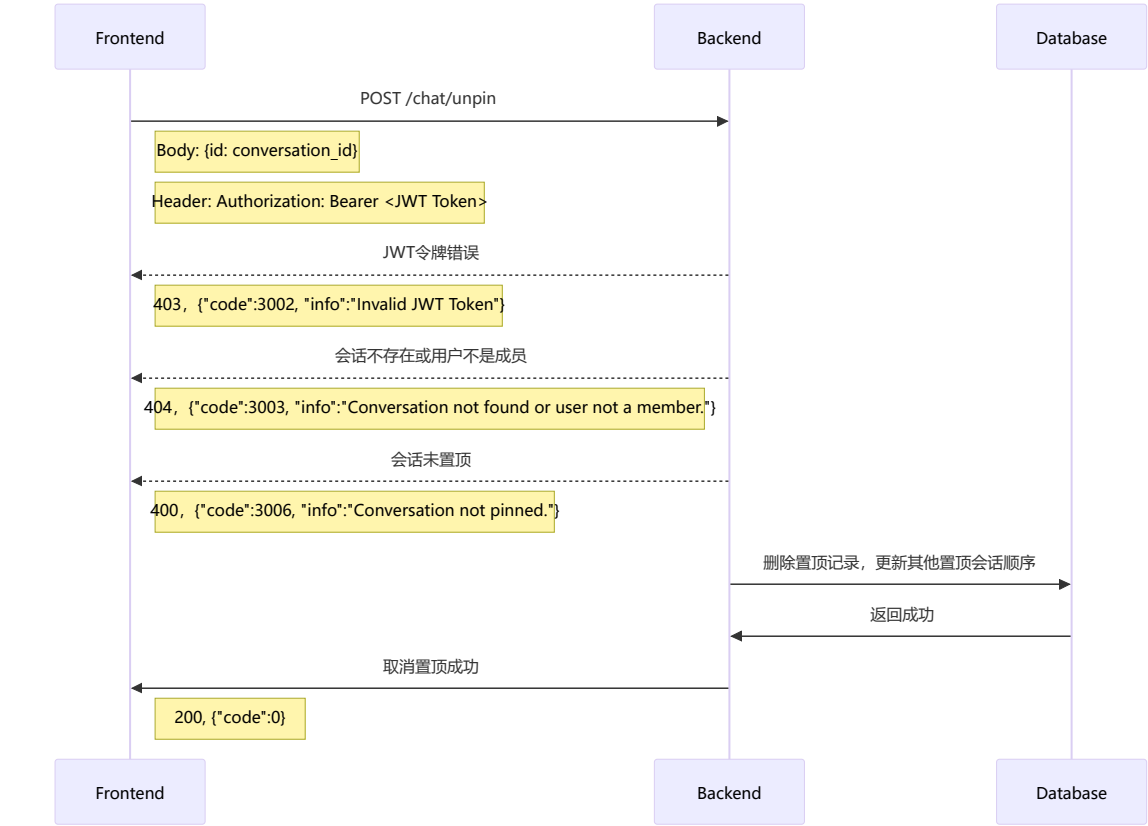
设置免打扰和置顶



置顶会话

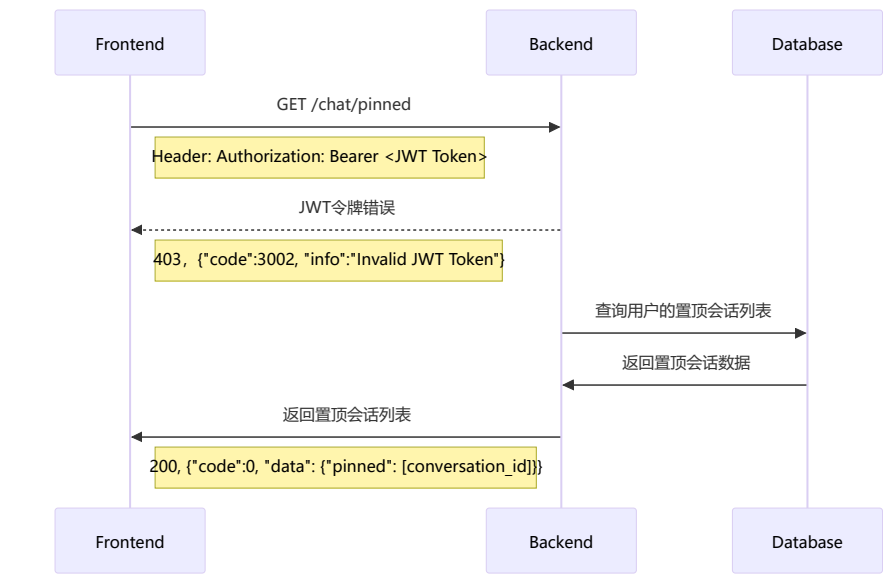


取消置顶会话

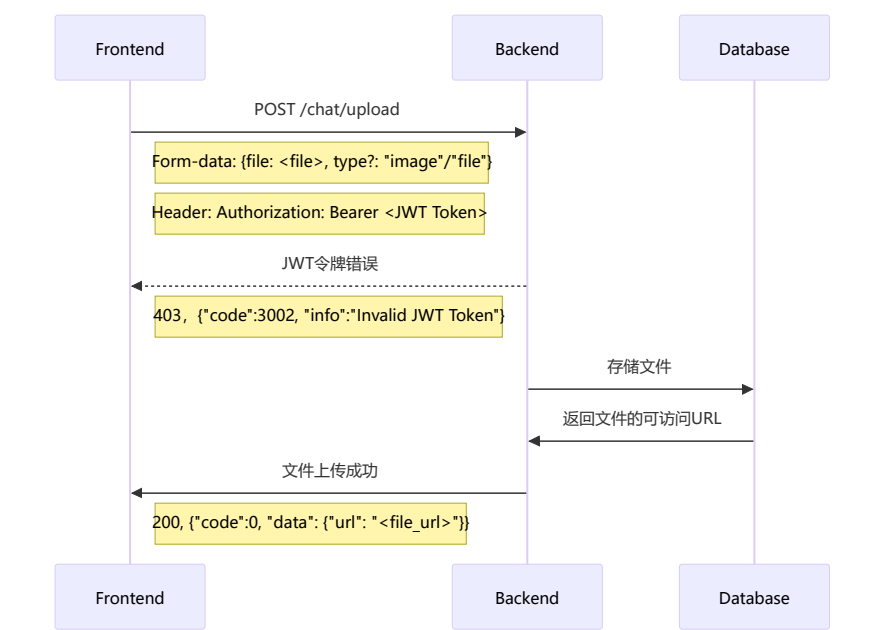


获取置顶会话列表



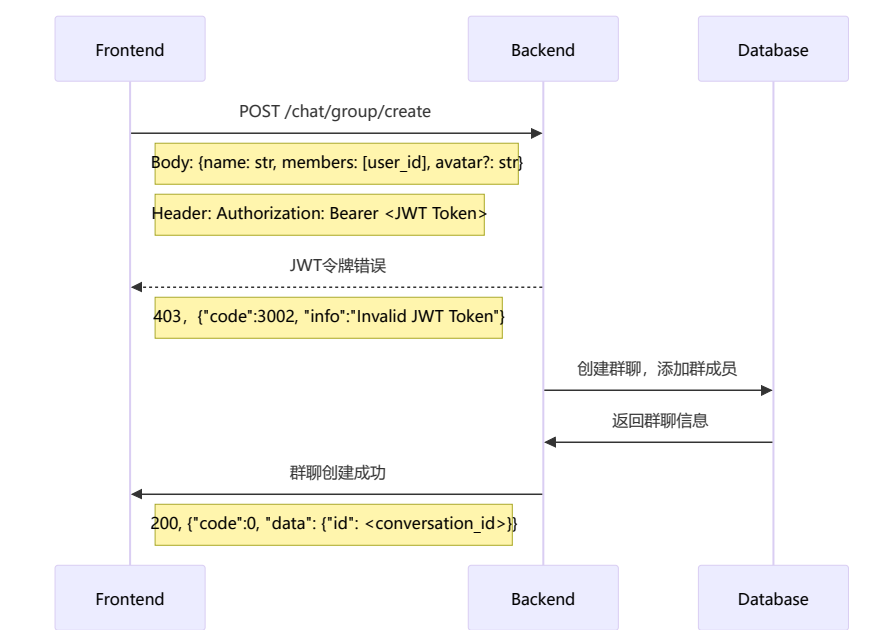


## 文件上传

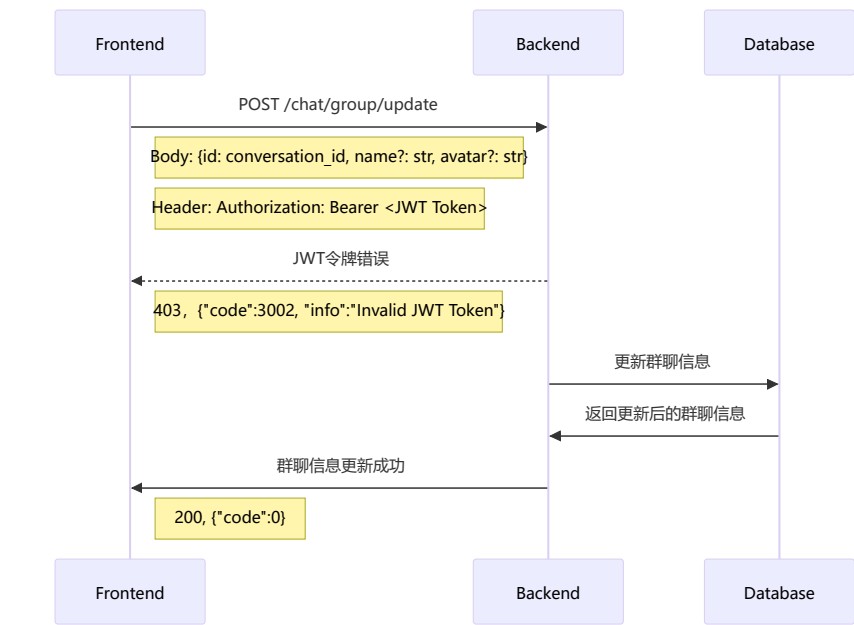


## 三、在线会话群聊部分（12分）

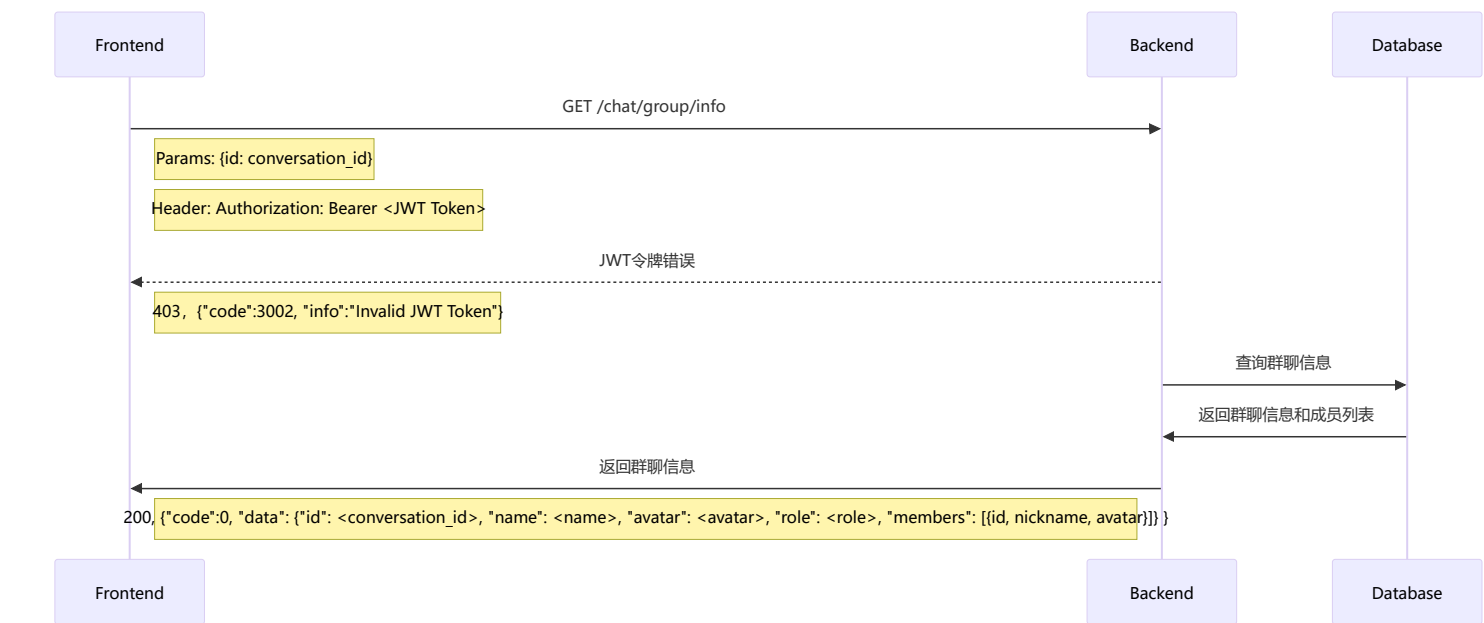
### 创建群聊



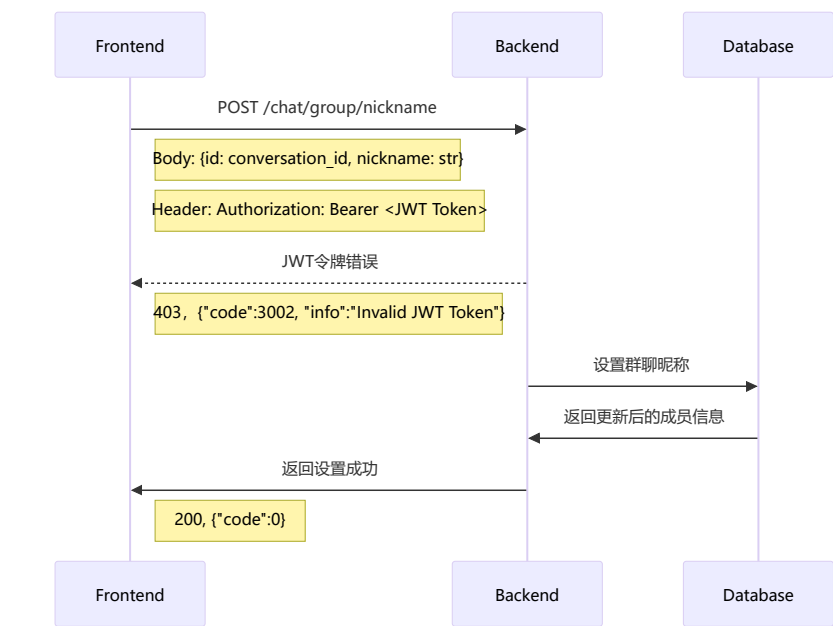
### 更新群信息



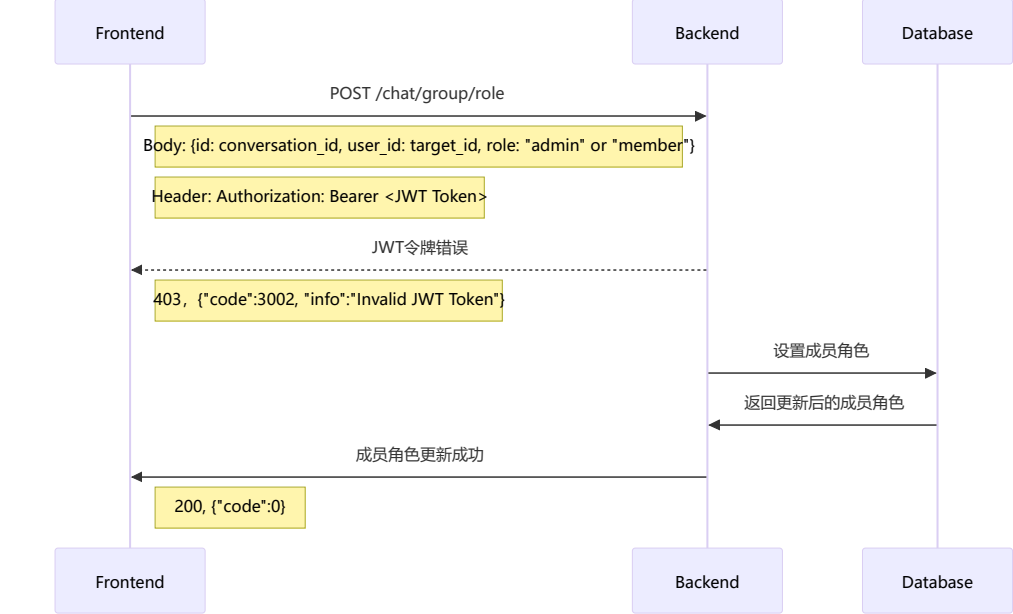
## 查询群聊信息



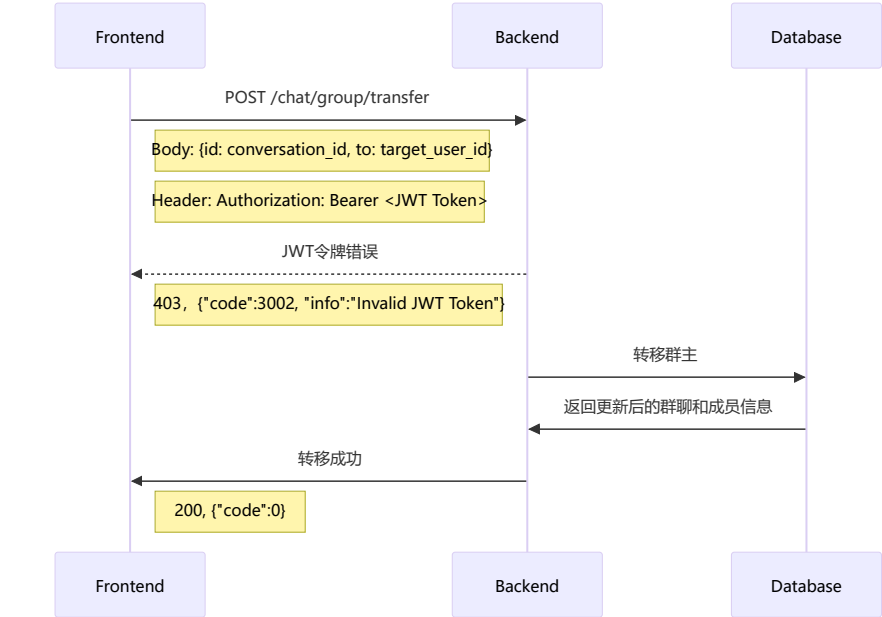
## 设置群聊昵称



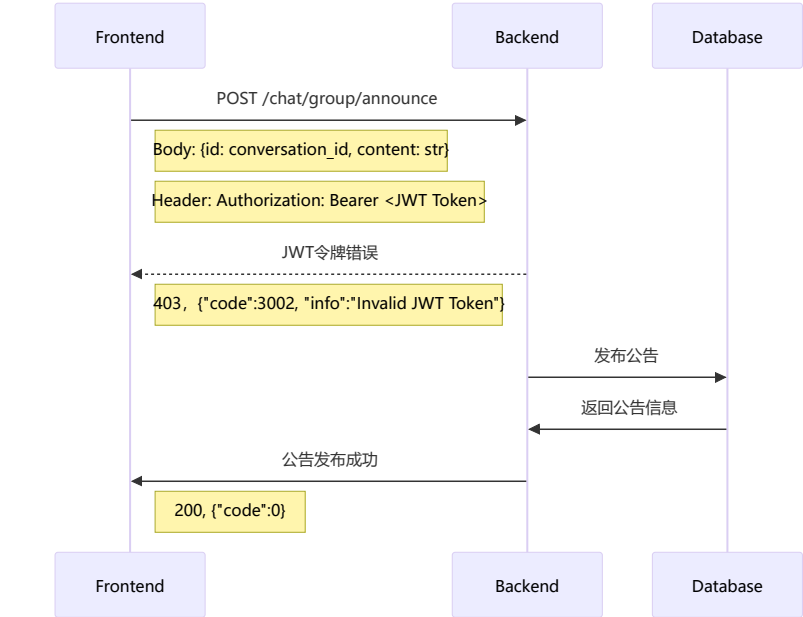
## 设置成员角色



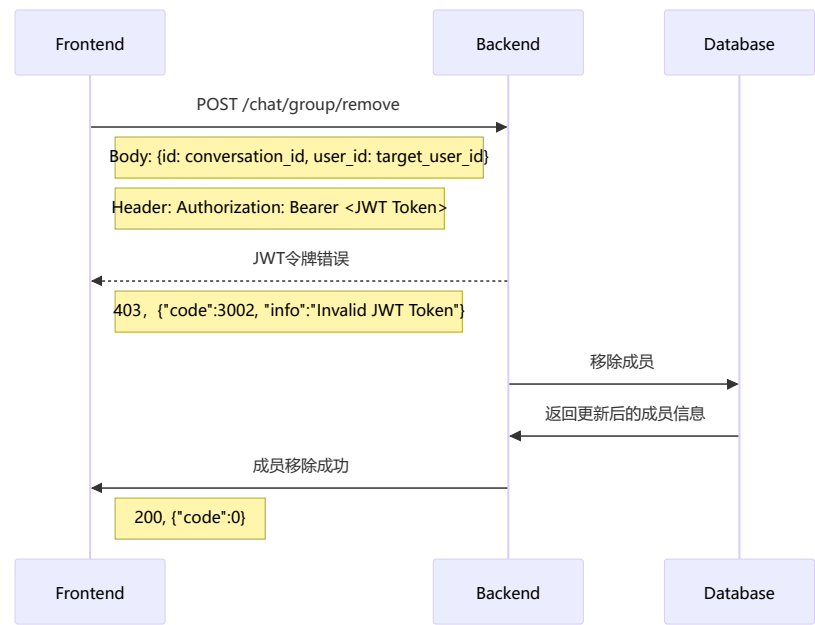
转移群主



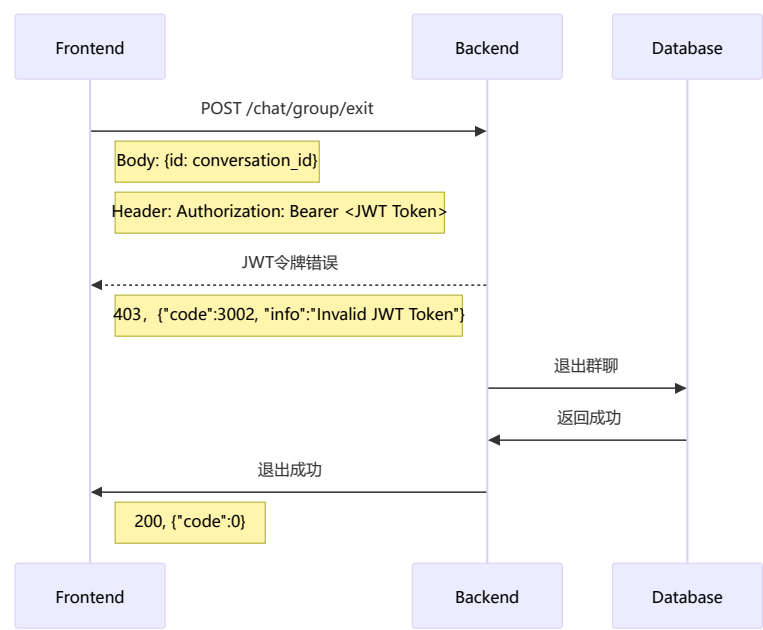
发布公告



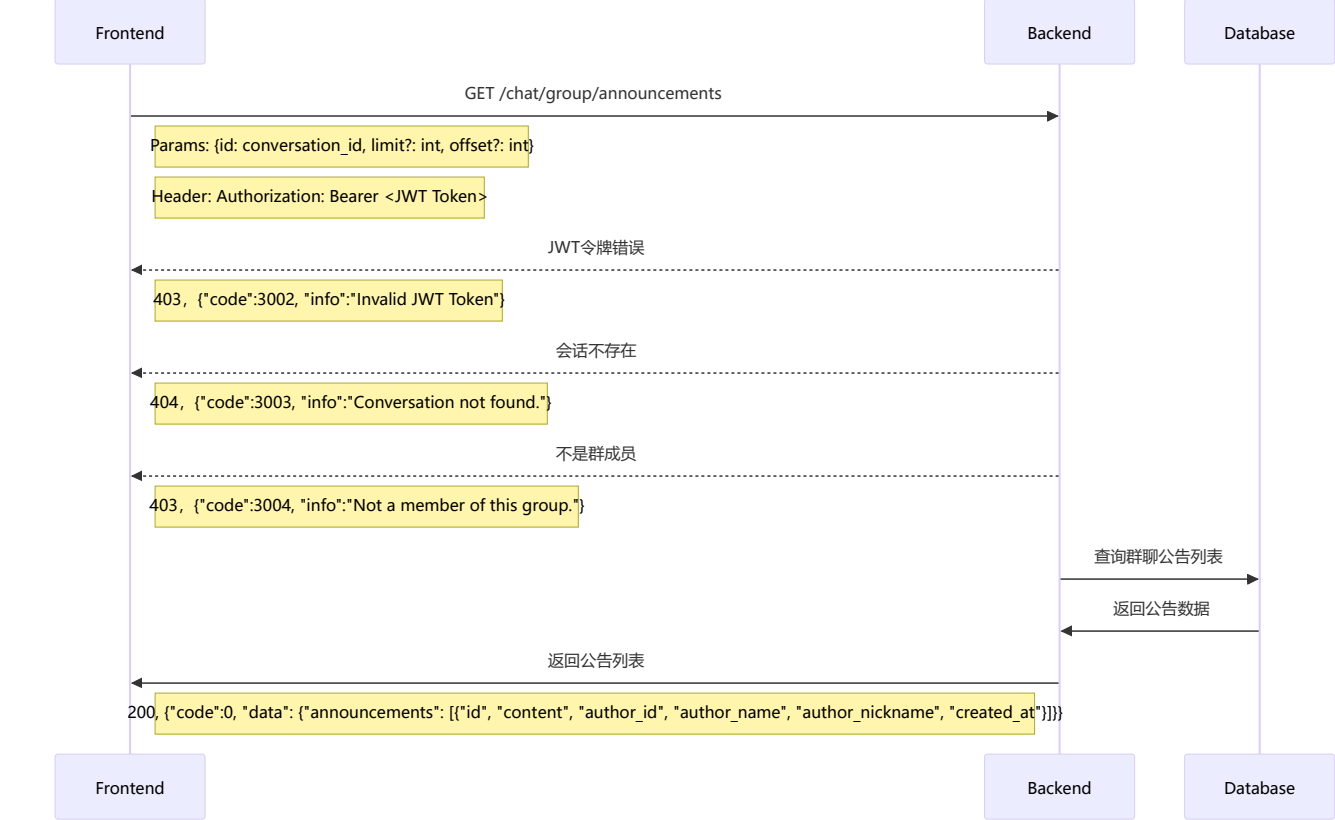
移除成员



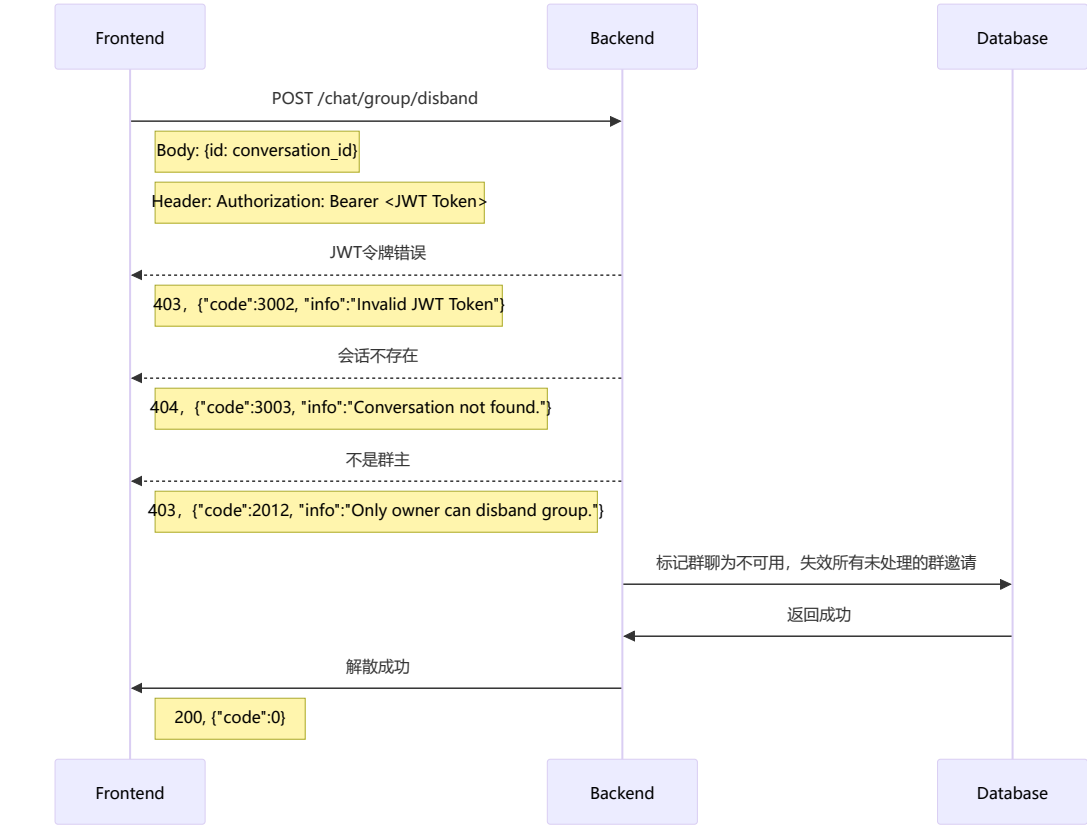
退出群聊



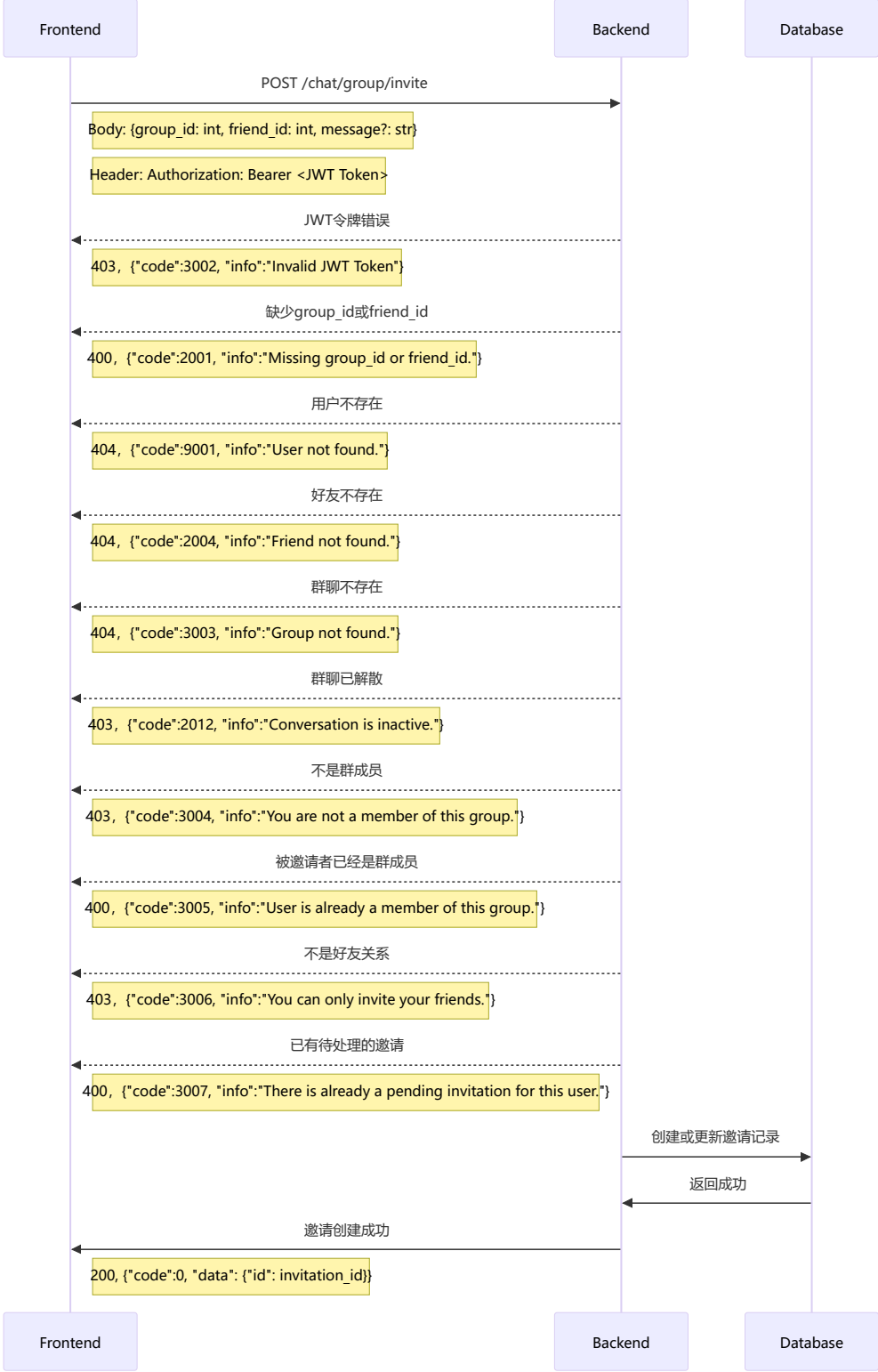
获取群聊历史公告



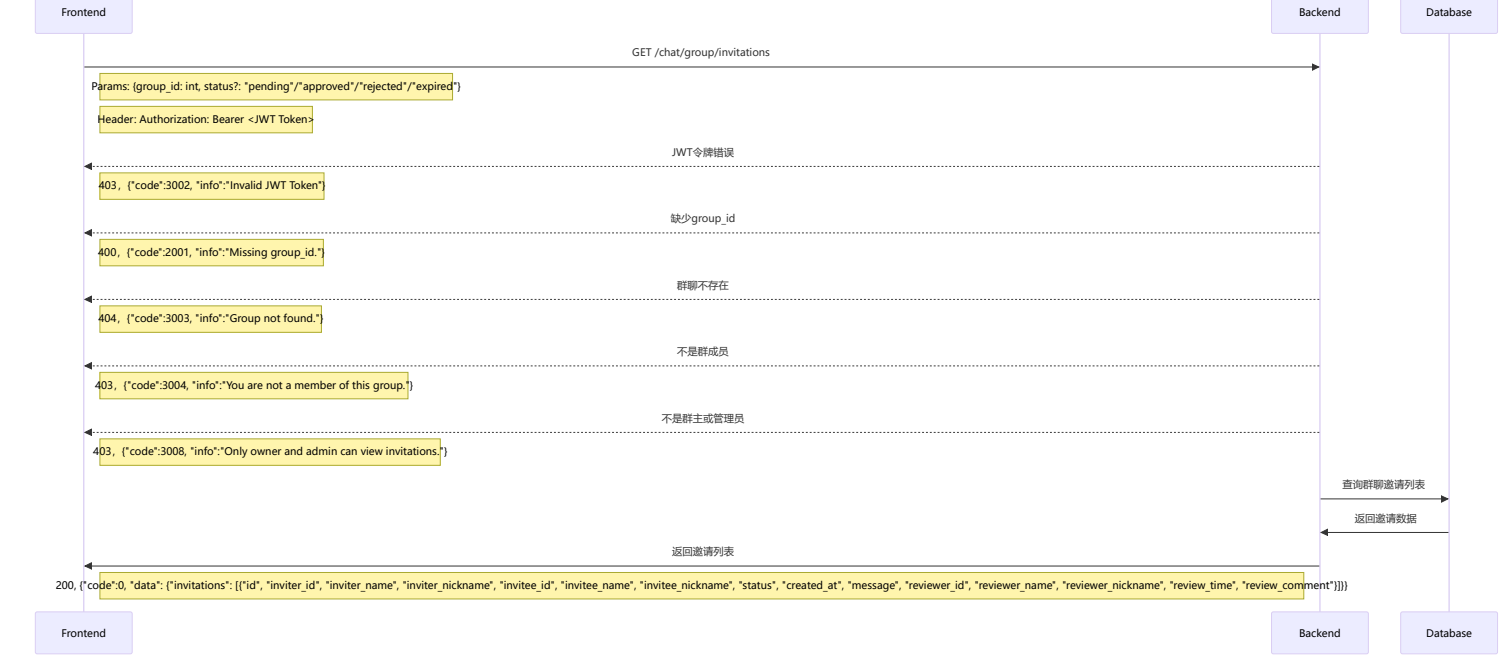
解散群聊



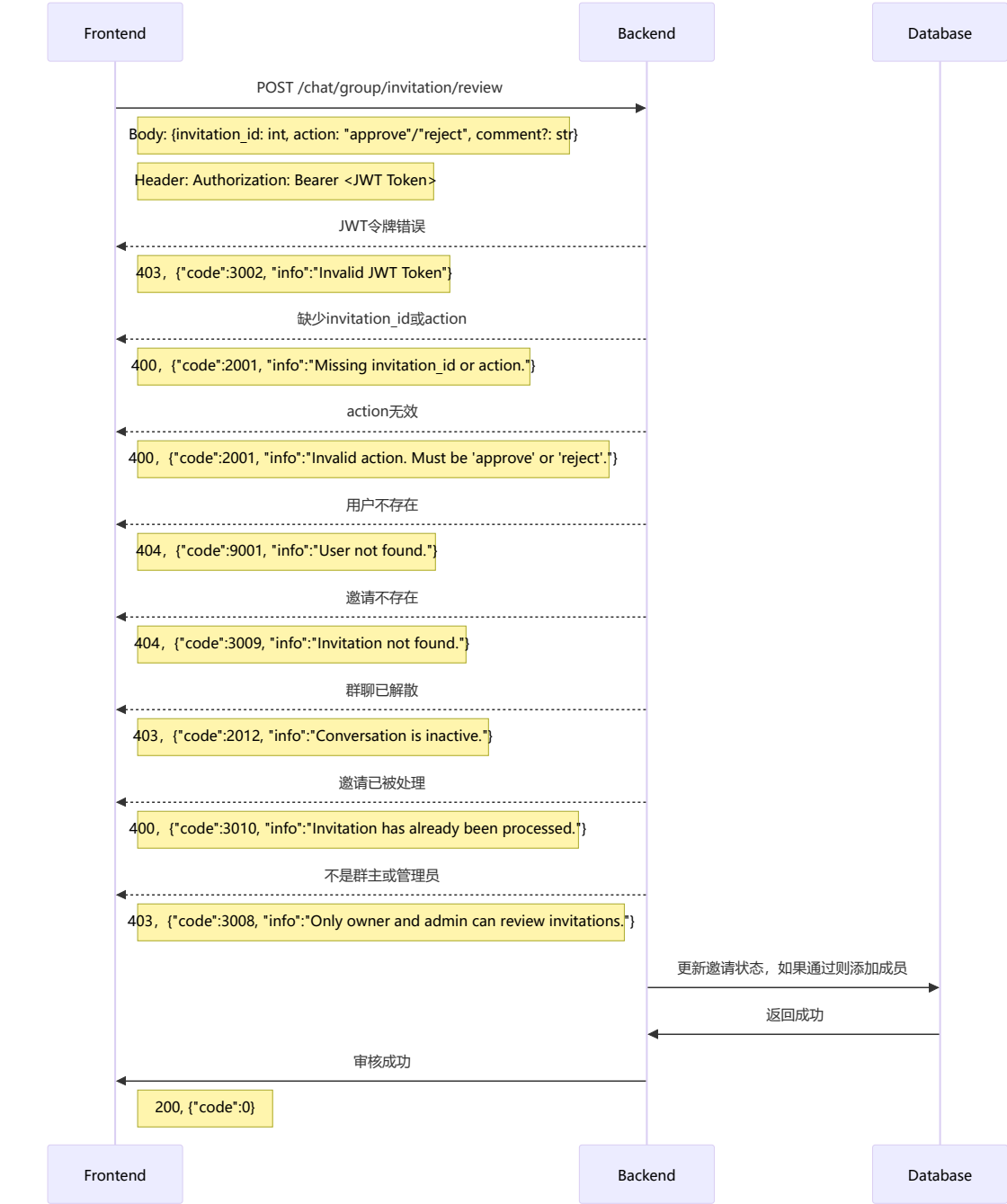
邀请好友加入群聊



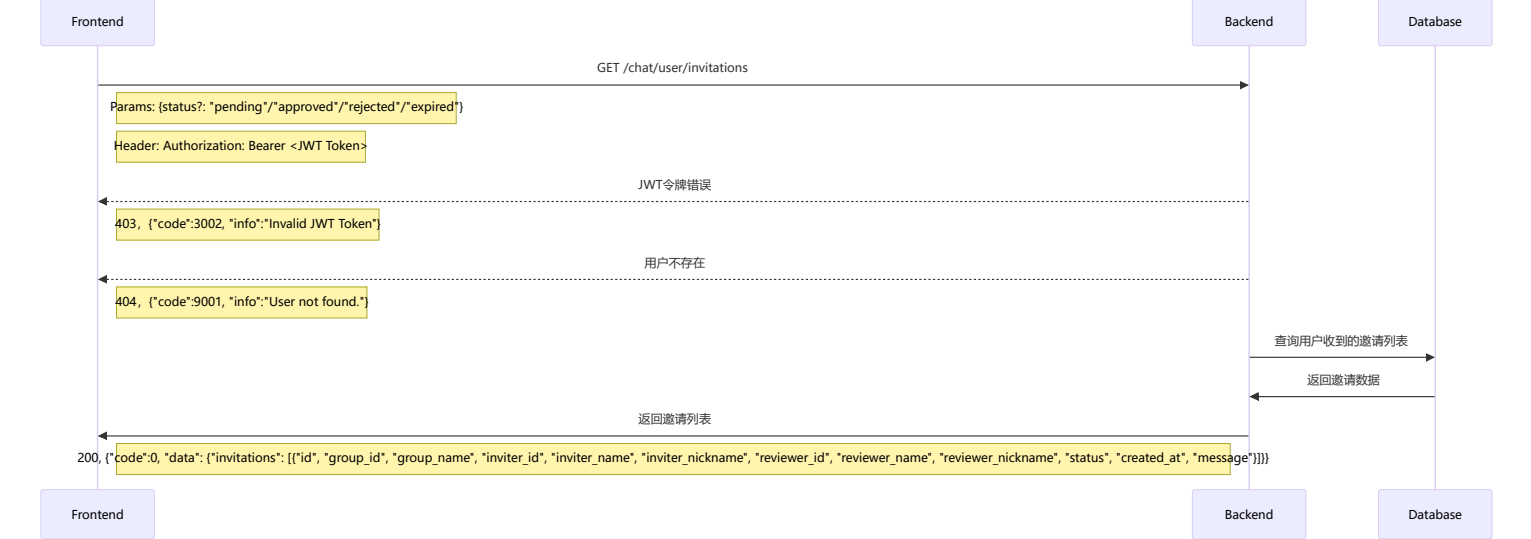
获取群聊邀请列表



审核群聊邀请

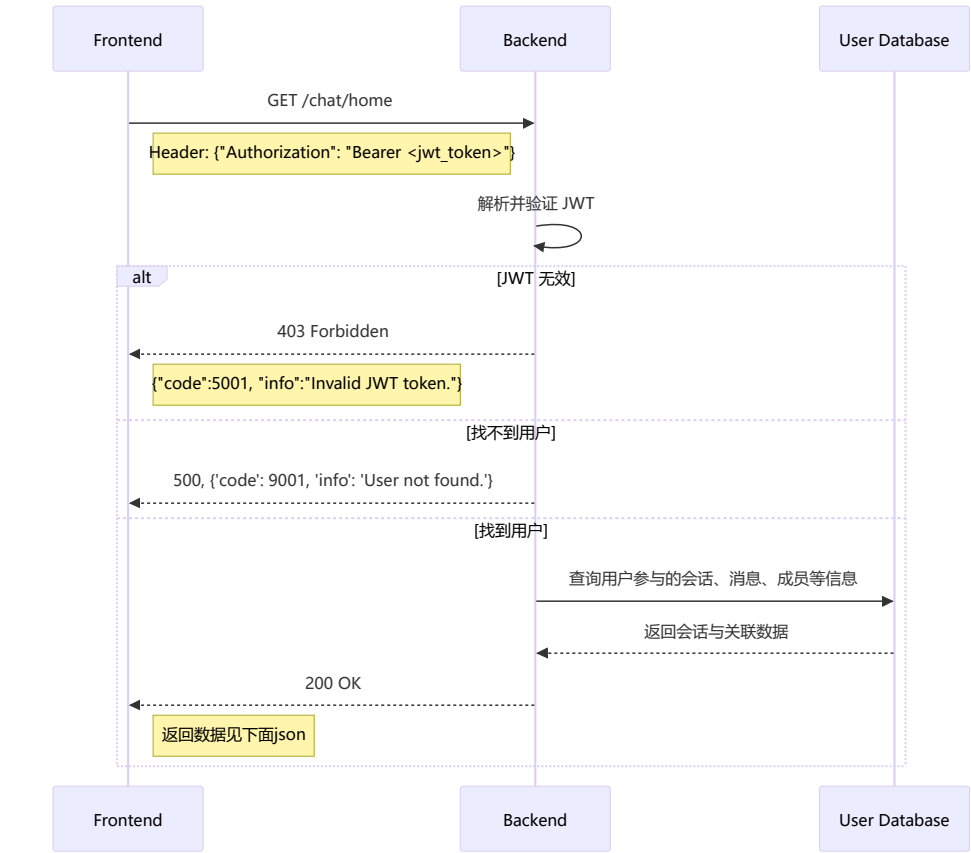


获取用户收到的群聊邀请列表



## 公用

### 显示主界面（请求会话信息）

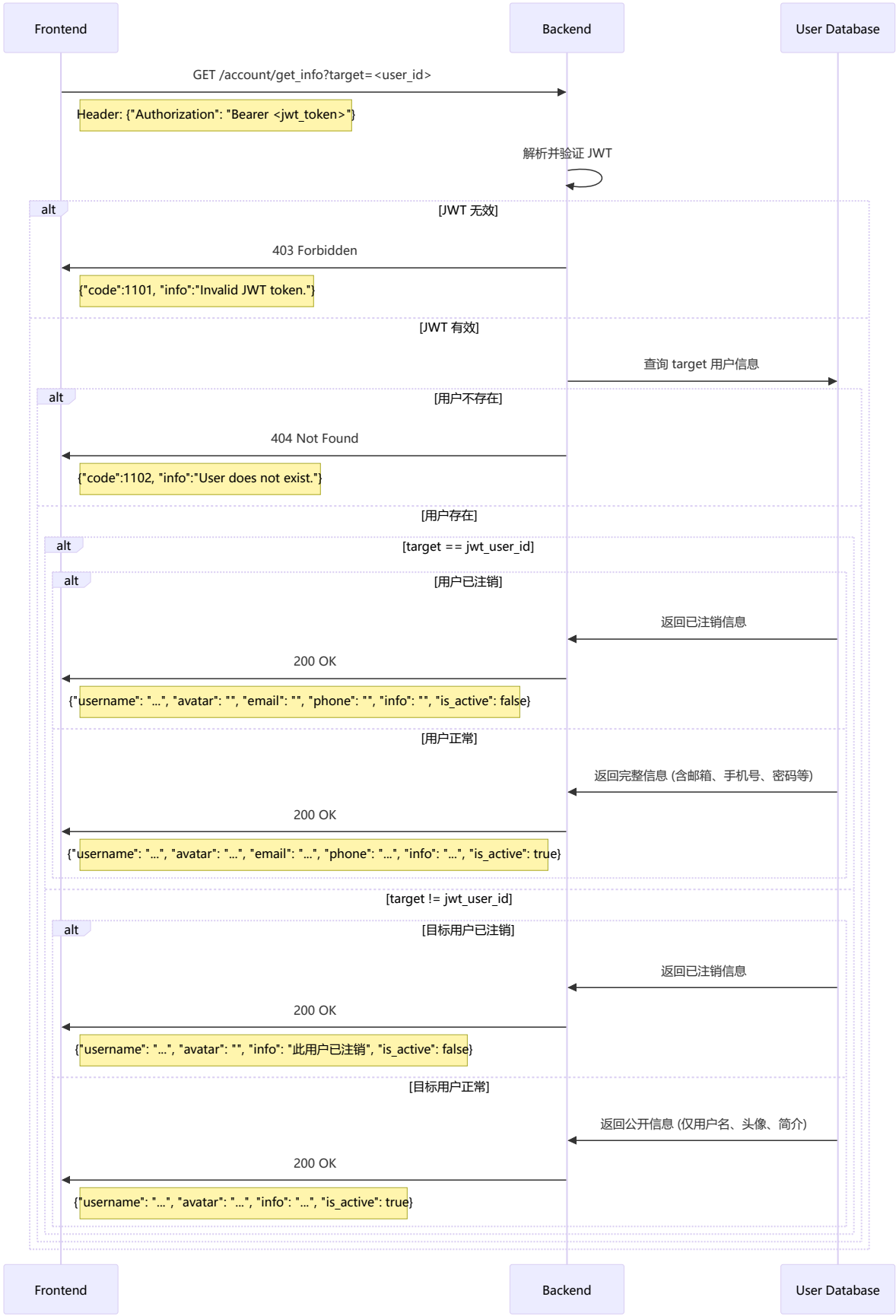


```
{
  "code": 0,
  "data": {
    "conversations": [
      {
        "id": 1,
        "type": "group",
        "name": "Python开发群",
        "avatar": "https://cdn.example.com/group_avatars/python.jpg",
        "unread_count": 3,
        "muted": false,
        "pinned": true,
        "members": [
          {
            "id": 1,
            "nickname": "alice",
            "avatar": "https://cdn.example.com/avatars/alice.png"
          }
        ]
      }
    ],
    "messages": [
      {
        "id": 101,
        "sender_id": 12,
        "content": "大家早上好！",
        "time": "2025-01-15T10:15:00Z",
      }
    ]
  }
}
```



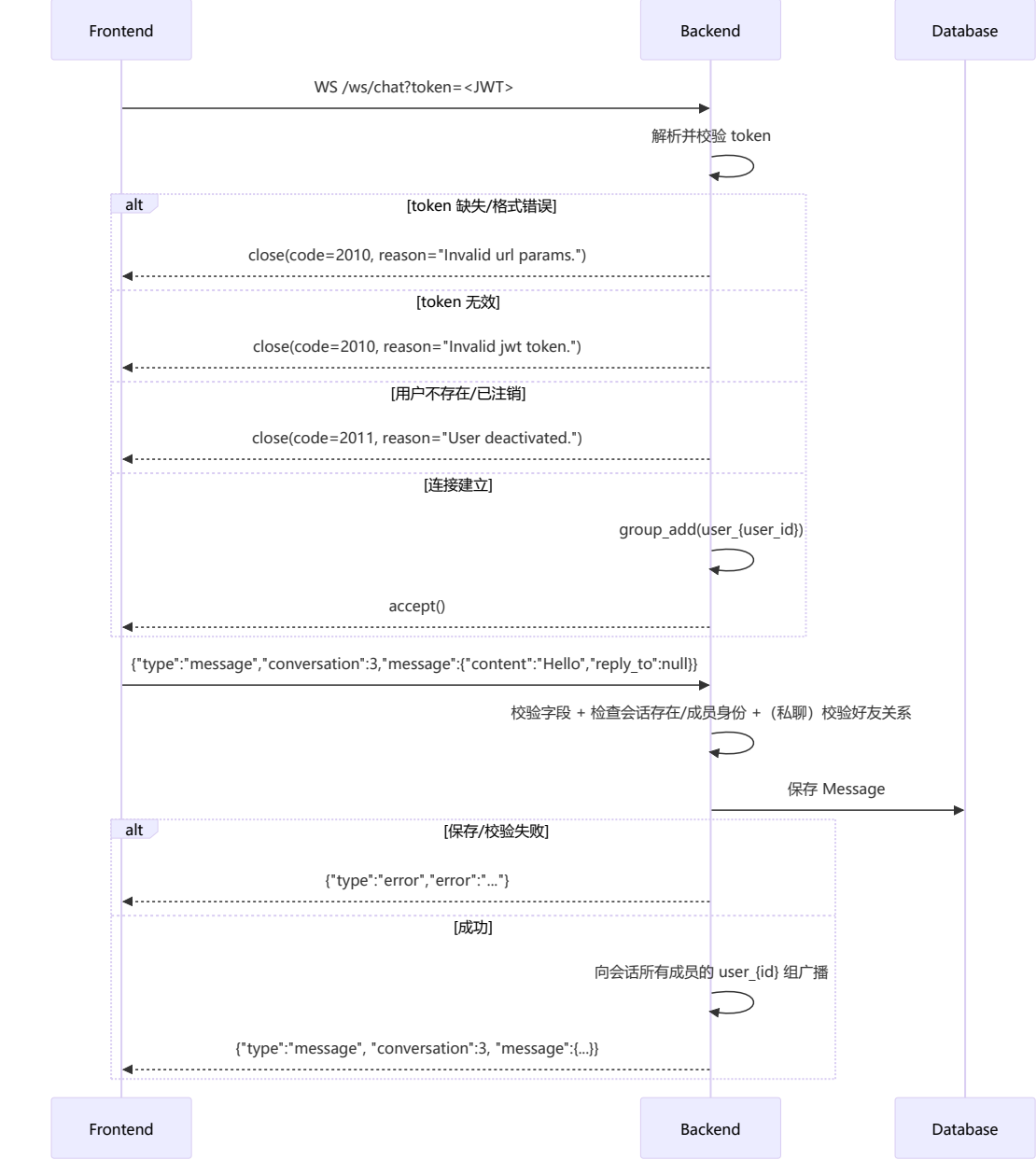
```
"type": "text",
"is_edited": false,
"valid": true,
"is_read": true
}
}
}
}
}
```

请求个人信息



发送消息

使用websocket实现



发送消息（客户端 -> 服务端）示例：

```
{
  "type": "message",
  "conversation": 3,
  "message": {
    "content": "...",
    "reply_to": null
  }
}
```

说明：后端当前 WebSocket 写库逻辑仅使用 content 与 reply\_to。如需发送图片/文件，通常先调用 POST /chat/upload 获得 URL，再将该 URL 放入 content。

服务端推送给客户端的消息（服务端 -> 客户端）格式：

```
{
  "type": "message",
  "conversation": 3,
  "message": {
    "id": 123,
    "sender": 7,
    "nickname": "Alice",
    "sender_nickname": "Alice",
    "content": "...",
    "reply_to": null,
    "reply_to_message": null,
    "time": "2025-01-01T00:00:00Z",
    "read_list": [7]
  }
}
```

备注：内部 channel event 名称为 chat\_message，但客户端收到的 type 为 message。

## 其他支持的 WebSocket 消息类型

1. 已读回执（客户端 -> 服务端）：

```
{ "type": "read_receipt", "conversation": 3, "message_id": 123 }
```

服务端广播（服务端 -> 客户端）：

```
{ "type": "read_receipt_update", "conversation": 3, "message_id": 123, "user_id": 7 }
```

2. 编辑消息（客户端 -> 服务端）：

```
{ "type": "edit_message", "conversation": 3, "message_id": 123, "content": "new text" }
```

服务端广播（服务端 -> 客户端）：

```
{ "type": "message_edited", "conversation": 3, "message_id": 123, "content": "new text", "user_id": 7 }
```

3. 撤回消息（客户端 -> 服务端）：

```
{ "type": "recall_message", "conversation": 3, "message_id": 123 }
```

服务端广播（服务端 -> 客户端）：

```
{ "type": "message_recalled", "conversation": 3, "message_id": 123, "user_id": 7 }
```

4. 会话事件（服务端 -> 客户端，仅推送）：

当发生“新会话创建、群相关变更、邀请待处理”等事件时，后端会通过用户组推送刷新提示：

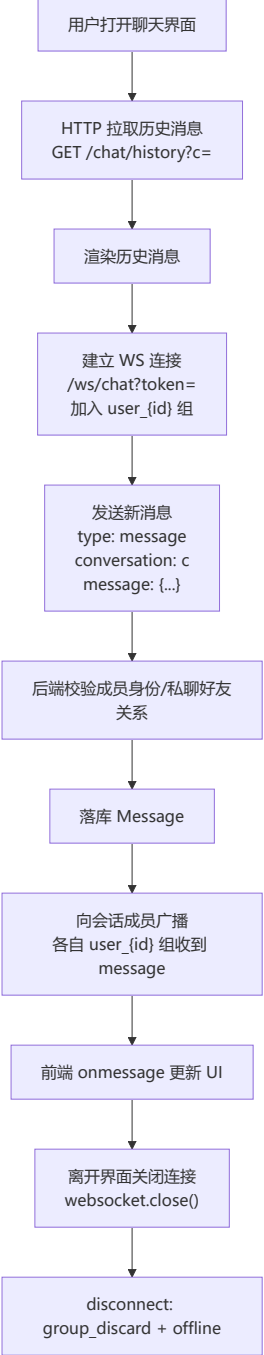
```
{ "type": "conversation_event", "event": "conversation_created", "conversation": 3 }
```

## 错误消息与当前行为说明

- JSON 解析失败：{ "type": "error", "error": "invalid\_json" }
- 消息类型非法：{ "type": "error", "error": "invalid\_message\_type" }
- 缺少必填字段：{ "type": "error", "error": "missing\_fields" }
- 用户/会话状态类错误：user\_not\_found / user\_deactivated / conversation\_not\_found / conversation\_inactive / not\_member / server\_error
- 私聊非好友：{ "type": "error", "error": "not\_friends" }（可能附带 message 文本提示）

注意：对某些“权限不满足”的场景（例如：编辑/撤回/已读操作时用户不是成员，或不是消息作者），当前实现可能不会返回任何 WebSocket 消息（表现为前端等待超时）。

## 发送消息全过程示意图



# Appendix

## 返回码一览

### HTTP返回码

| 200 | OK                 |                  |
|-----|--------------------|------------------|
| 201 | Created            | POST创建新数据        |
| 202 | Accepted           | 请求成功，但还未执行（用于异步） |
| 302 | Found              | 重定向              |
| 400 | Bad Request        | 请求格式错误           |
| 401 | Unauthorized       | 身份未认证            |
| 403 | Forbidden          | 身份已认证，但没有权限      |
| 404 | Not Found          | 资源不存在            |
| 405 | Method Not Allowed | 请求方法错误           |

### WebSocket Close Code

- 2010：WS 连接参数不合法（缺少 token / query 解析失败）或 JWT 无效
- 2011：用户已注销（或用户不存在）

## 业务返回码

- -3: Bad method.
- 0: 成功 (成功响应固定包含 `info="Succeed."`, 其余字段由接口决定)
- 注册/登录
  - 1001: 请求体缺少 `username` 或 `password` (或解析失败)
  - 1002: 用户名已被占用
  - 1003: 用户名不存在
  - 1004: 密码错误
  - 1005: 用户名格式不合法 (注册) / 用户账号已注销 (登录时返回 `User account has been deactivated.`)
  - 1006: 密码格式不合法 (注册)
- 用户信息
  - 1101: JWT 无效 (`Invalid JWT token.`)
  - 1102: 用户不存在 (`User does not exist.`)
- 修改个人信息
  - 1201: JWT 无效 (`Invalid JWT Token.`)
  - 1202: 用户不存在 (`User does not exist.`)
  - 1203: 请求体不合法/缺少字段
  - 1204: `field` 不合法
  - 1205: 修改敏感字段时原密码错误
  - 1206: 新密码格式不合法
- 用户注销
  - 1301: JWT 无效
  - 1302: 用户不存在
  - 1303: 请求体缺少 `password`
  - 1304: 密码错误
- 会话/群聊/消息 (HTTP)
  - 2001: 请求体/参数不合法 (例如 `role/action` 不合法等)
  - 2004: 目标用户不存在 / 好友不存在 (依接口场景)
  - 2005: 涉及“已注销用户”的群聊操作被拒绝 (例如: 不能邀请/转让/设管理员等)
  - 2006: 不能与已注销用户私聊
  - 2012: 会话不可用或无权限 (例如: 会话已失效、非群主/管理员、或角色约束不满足等)
  - 2013: 用户账号已注销 (`User account is deactivated.`)
- 会话历史/置顶/邀请
  - 3001: 缺少 `Authorization` (`Authorization failed.`)
  - 3002: JWT 无效 (`Invalid JWT Token.`)
  - 3003: 会话/群聊不存在, 或用户不是成员
  - 3004: 成员不存在 / 目标不在会话中 / 非群成员等
  - 3005: 会话已置顶 / 已是群成员 (依接口场景)
  - 3006: 会话未置顶 / 只能邀请好友 (依接口场景)
  - 3007: 已存在待处理邀请
  - 3008: 只有群主/管理员可查看或审核邀请
  - 3009: 邀请不存在
  - 3010: 邀请已被处理
- 好友与好友分组
  - 4001: JWT 无效 (部分好友分组接口也复用该码表示请求体缺少 `group_id/friend_id`)
  - 4002: 已经是好友
  - 4003: 目标用户不存在
  - 4004: 已存在待处理好友申请
  - 4005: 好友申请不存在
  - 4006: 不能对自己发起好友操作 (加好友/查好友等)
  - 4007: 好友关系不存在
  - 4008: 不是好友关系
  - 4009: 不能添加已注销用户为好友
  - 4010: 不能与已注销用户建立好友关系 / 好友分组名为空 (依接口场景)
  - 4011: 好友分组名已存在 / 目标用户已注销 (依接口场景)
  - 4012: 处理好友申请时存在已注销用户 / 好友不存在 (依接口场景)
  - 4013: 不能删除已注销用户的好友关系 / 不是好友关系 (依接口场景)
  - 4014: 分组不存在 / 不能将已注销用户加入分组 (依接口场景)
  - 4015: 好友已在分组中 / 好友不在分组中
  - 4016: 分组不存在 (`rename`)
  - 4017: 分组不存在 (`delete`)
- 其他
  - 5001: 服务器内部失败 (例如创建 `Pending` 失败; 部分场景也复用该码表示 JWT 无效)
  - 9001: 用户不存在 (通常表示鉴权成功但数据库记录异常等)